

The Man-Bites-Dog Phenomenon in Cybersecurity: A Case Study on Hardening Legacy Hardware Communications with the Sony AIBO ERS-7

Alexis Regardin



Master thesis submitted under the supervision of
Prof. Jan Tobias Mühlberg

the co-supervision of
Adelin Roty

in order to be awarded the Degree of
Master in Cybersecurity with focus on Cryptanalysis
and Forensics

Academic year
2025-2026

Title: The Man-Bites-Dog Phenomenon in Cybersecurity: A Case Study on Hardening Legacy Hardware Communications with the Sony AIBO ERS-7

Author: Alexis Regardin

Master in Cybersecurity with focus on Cryptanalysis and Forensics

Academic year: 2025-2026

Abstract

Critical infrastructure in the industrial and healthcare sectors frequently relies on legacy hardware that is functionally adequate but cryptographically obsolete. When hardware replacement is financially or logistically unfeasible, security must be retrofitted. This thesis investigates the hardening of remote communication protocols in legacy embedded systems using the Sony Artificial Intelligence RoBOt (AIBO) ERS-7 robot as a primary case study. Released in the early 2000s, the ERS-7 features a 64-bit Reduced Instruction Set Computer (RISC) processor and 64 MB of Random-Access Memory (RAM), yet its wireless capabilities are restricted to the deprecated Wired Equivalent Privacy (WEP) protocol. Since the link-layer security cannot be updated, this research focuses on implementing an application-layer cryptographic wrapper to ensure mutual authentication, data integrity and confidentiality. Using the OPEN-R Software Development Kit (SDK) and C++, this study addresses the challenge of implementing modern cryptographic primitives within the constrained memory and processing environments of a legacy IoT device. The project evaluates the trade-off between security robustness and real-time performance, providing a framework for securing similar legacy Internet of Things (IoT) devices or cyberphysical systems where traditional network-level protections are no longer viable.

Keywords: Legacy Hardware - Robotic Security - Remote Communication - Embedded Cryptography

Acknowledgements

I would like to express my deepest thanks to Professor Mühlberg for agreeing to be my supervisor for this Master's thesis topic, which strays somewhat from the beaten track. I would like to thank him again for advising me on how to turn a project into a proper thesis topic.

Many thanks to Adelin Roty for agreeing to be my co-supervisor and for his advice in helping me to develop a clear and detailed assessment method.

Of course, a big thank you to all the teachers I have had throughout my life, who have all, each in their own way, enabled me to acquire the knowledge and methodology needed to complete this work.

Next, and of course, a million thanks to my friends for taking the time to listen to me ramble about a 20-year-old robot, but one which, "if I get it right, is really interesting".

Similarly, thank you to my family for supporting me while I worked on this subject (thanks also for the Clié, it was neither easy nor cheap to find).

Finally, thanks to my father for bringing home that robot dog one day more than 10 years ago - the one that had been gathering dust in a closet. I will not say that this was what sparked my absolute passion for computer science and subsequently led me to do this Master's degree in cybersecurity (I am not one of those pre-ordained geniuses, nor even a genius at all) but it certainly influenced me to some extent.

To anyone reading this paper, I hope you will find it interesting and that it inspires you to have a rummage through your old appliances, either to give them a new lease of life, or to donate them or have them recycled.

Preface

Securing legacy systems is like trying to put rebar into jello without changing its appearance or flavor.

Eugene Spafford [1]

Choosing a subject for a master thesis is no easy task. More often than not, a subject will be about recent evolutions in the designated field (even more true in computer science and cybersecurity which are rapidly evolving fields).

But then, why focus on legacy systems?

The reason might be the fact that, despite what marketing would have us believe, there are nowadays critical systems still built on legacy technology - developed by people who are either retired or long gone - that keep the world turning (COBOL, SCADA and old IBM mainframes to cite a few).

It could also lie in the fact that we are living at a historic juncture for humankind where we are confronted with the consequences of so many years of pollution and where, despite everything, we continue to produce more and more, faster and faster, when we should instead be slowing down, reusing and recycling. And this sort of adaptation should also involve updating legacy systems rather than seeking to replace them (why throw away 'old' mobile phones – which are more powerful than some computers from a few years ago – and not repurpose them when they are still fully functional? [2]).

Or perhaps simply because it was a topic that seemed interesting.

Usage of LLM and online tools

For the purpose of this master thesis, multiple tools were used. DeepL was used to translate and rephrase into English. Google Scholar, ScienceDirect-Elsevier and ResearchGate were used to find relevant documentation, research papers, data and to obtain appropriate citations. Zotero was used to regroup relevant documentation and to generate a pre-formatted bibliography.

Two LLMs were used: Google's Gemini 3.1 Pro and Anthropic's Claude Opus 4.8. Those were used:

- As personal assistants, to provide feedback and criticism with the goal of improving language and structure, to correct spelling, grammar and sentence structure errors, and to ensure the same style of English (British) is used.
- To help identify hidden bugs and crashes related to the quirks inherent in OPEN-R.
- To refine the search for documents when standard tools failed to produce satisfactory results. The results obtained were then verified and never accepted at face value.
- To implement the testing and measurements Python script based on the necessary requirements.

Table of Contents

Abstract	I
Abstract	I
Table of Contents	VI
List of Figures	VI
List of Tables	VI
List of Abbreviations	VI
1 Introduction	1
1.1 Context	1
1.1.1 Presentation of Sony’s AIBO ERS-7	2
1.2 Problem statement	3
1.3 Structure and contributions	5
2 Background and related work	7
2.1 Key features and functionalities of OPEN-R	7
2.1.1 TinyFTPD	7
2.1.2 OPEN-R objects	8
2.1.3 Inter-objects communication	8
2.1.4 Core Class Lifecycle	9
2.1.5 Aperios Network Toolkit (ANT) Messaging	10
2.1.6 stub.cfg	11
2.1.7 Stubgen2	11
2.1.8 Object memory allocation	11
2.1.9 Makefile and linking	11
2.1.10 Debugging	12
2.2 Related work and literature	12
2.2.1 Securing legacy robots	13
2.2.2 Security strategies on legacy systems	14
3 Practical work and implementation	17
3.1 WEP security – or rather, the lack of it	17
3.1.1 Telnet console	18
3.2 Setup and necessary assets	18
3.2.1 OPEN-R SDK	18
3.2.2 Programmable Memory Stick	20
3.3 Threat modelling	23
3.3.1 External entities	24
3.3.2 Process	24
3.3.3 Data Flow	24
3.3.4 Data Store	25

3.4	Risk analysis	26
3.4.1	High-priority threats (immediate mitigation)	26
3.4.2	Medium-priority threats (planned mitigation)	27
3.4.3	Low-priority and accepted risks	27
3.4.4	Summary using a risk scoring matrix	28
3.5	TinyConsole	28
3.5.1	Reasoning and objective	28
3.5.2	Unauthenticated XOR prototype	29
3.5.3	ChaCha20 and Poly1305	32
3.5.4	Choice of the implementation	33
3.5.5	Asymmetric Authentication with Ed25519	34
3.5.6	Secure TinyConsole implementation	35
3.6	ROS 2 integration	46
3.6.1	Usefulness and choice	46
3.6.2	Security considerations	46
3.6.3	Building blocks	47
3.6.4	Keyboard remote control	48
3.7	Final check of the Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege (STRIDE) threats status	50
4	Testing and measurements	51
4.1	Testing	51
4.2	Measurements	54
4.2.1	Verification of the nonce construction	55
4.2.2	Ciphertext analysis via ent	56
4.2.3	Memory footprint	56
4.2.4	Latency comparison	57
5	Discussion & Conclusion	60
5.1	Comparison with state of the art and related work	60
5.2	Lessons learned: How to secure legacy systems?	61
5.2.1	Taking it a step further: how can a system be made future-proof?	63
5.3	Limitations of validity	65
5.4	Future work	67
5.4.1	Sequels to TinyConsole	67
5.4.2	Applying the methodology to other legacy systems	68
5.5	Conclusions	68
5.5.1	Open-source	70
5.5.2	The Man-Bites-Dog Phenomenon	70
	Bibliography	75
	Appendix	76
1	TinyConsole Makefile	76
2	DoInit and DoStart	76

3	Assignment of joint values	77
---	--------------------------------------	----

List of Figures

1.1	AIBO ERS-7	2
2.1	Ghost code samples	7
2.2	Inter-object communication as represented in the Programmer's Guide	8
2.3	Core class as represented in the Programmer's Guide	9
2.4	Object communication with IPStack as represented in the Programmer's Guide	10
3.1	Sony CLIE PEG-TJ25	21
3.2	Initial AIBO data flow diagram	23
3.3	Memory Stick Media	26
3.4	Risk scoring matrix (adapted from Mees [3])	28
3.5	Sequence diagram of TinyConsole execution	31
3.6	Locations of the three types of leg joints	38
3.7	Skitter UI	39
3.8	Sequence diagram of secure TinyConsole	45
3.9	Final AIBO data flow diagram	50
4.1	Replay mode	52
4.2	Tamper mode	53
4.3	Interception of keys mode	53
4.4	Connection reset	54
4.5	Packet capture of a single PING exchange	58
4.6	Histogram of the Round-Trip Time (RTT) of the three versions of TinyConsole	59

List of Tables

3.1	STRIDE per element	24
4.1	Comparison of packaged .BIN and unpackaged .bin sizes and cryptographic overhead	57
4.2	Comparison of source code footprint and cryptographic overhead	57
4.3	Round-trip latency of one PING command over the TinyConsole link	57
4.4	Round-trip latency differences when the motor command-vector allocation is removed.	58

List of Abbreviations

AEAD	Authenticated Encryption with Associated Data. 5, 32, 35, 36, 40–44, 50, 52, 60, 61, 65, 69, 70
AES	Advanced Encryption Standard. 16
AIBO	Artificial Intelligence RoBOt. 1–3, 5, 67, I, IV, VI
ANT	Aperios Network Toolkit. 9–11, 30, 35, 67, IV
AP	Access Point. 23, 51
API	Application Programming Interface. 7, 8, 21, 29, 36, 42
ARM	Advanced RISC Machine. 16, 21
ATM	Automated Teller Machine. 1
CMOS	Complementary Metal-Oxide-Semiconductor. 3, 67
CPU	Central Processing Unit. 12, 15, 19, 21, 25, 51
CRQC	Cryptographically Relevant Quantum Computer. 33
CSPRNG	Cryptographically Secure Pseudorandom Number Generator. 40
DCS	Distributed Control Systems. 4
DDS	Data Distribution Service. 50
DH	Diffie-Hellman. 62, 67, 70
DMCA	Digital Millennium Copyright Act. 2
DNS	Domain Name System. 29
DoS	Denial of Service. 27, 54, 66
EDR	Endpoint Detection and Response. 61
FAT	File Allocation Table. 21
FTP	File Transfer Protocol. 7
IDE	Integrated Development Environment. 19
IoT	Internet of Things. I
IP	Internet Protocol. 7, 10, 30, 54
LAN	Local Area Network. 7, 14, 18, 62, 69
LLM	Large Language Model. 62, III
MAC	Message Authentication Code. 16, 33, 61
MDS	Mobile Dexterous Social. 13, 14
MIPS	Microprocessor without Interlocked Pipelined Stages. 2, 12, 19, 28, 32
MITM	Man-in-the-middle. 40, 51, 69
MRI	Magnetic Resonance Imaging. 1, 68
MSB	Most Significant Bit. 40, 44
NaCl	Networking and Cryptography library. 34, 42

NIST	National Institute of Standards and Technology. 32, 63, 66
OS	Operating System. 1, 2, 4, 6, 18–21, 23, 29, 36, 40, 41, 43, 49, 51, 62, 63, 69
PDA	Personal Digital Assistant. 20, 21
PLC	Programmable Logic Controllers. 4, 14, 63
PMS	Programmable Memory Stick. 17, 20, 21
PQC	Post-Quantum Cryptography. 33
RAM	Random-Access Memory. 51, I
RISC	Reduced Instruction Set Computer. I
RNG	Random Number Generator. 32, 36, 40, 61, 67
ROS	Robot Operating System. 13, 46
RTT	Round-Trip Time. 55, 57–59, 61, 67, 69, VI
SCADA	Supervisory Control And Data Acquisition. 4, 14
SDK	Software Development Kit. 1, 5, 18–20, 29, 61, 69, 70, I
SDRAM	Synchronous Dynamic Random-Access Memory. 2
SHA	Secure Hash Algorithm. 34
STRIDE	Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege. 17, 23, 24, 50, 51, 62, V, VI
TCP	Transmission Control Protocol. 7, 10, 24, 25, 27–30, 37, 39, 40, 42, 44
TLS	Transport Layer Security. 16
UDP	User Datagram Protocol. 10, 13, 29
URBI	Universal Robotic Body Interface. 5
VM	Virtual Machine. 19
WEP	Wired Equivalent Privacy. 3, 17, 18, 23, 62, 69, I, IV
WPA	Wi-Fi Protected Access. 17, 18

Chapter 1

Introduction

A legacy system can be defined as a "*software or hardware that was built using methods that are outdated or obsolete*" (per the Dictionary of Archives Terminology [4]). This is a very broad definition for a subject that encompasses a wide array of technology and systems (both software and hardware).

It is not unusual for those systems to be proprietary and solely developed or sold by a company that does not exist anymore. What those systems do have in common is their status: obsolete.

In terms of cybersecurity, this entails that those systems are not secure anymore (if they ever were). Given their nature, it would be difficult, if not futile, to attempt to propose a single security standard or procedure for these systems. First of all because an industrial robot, a Magnetic Resonance Imaging (MRI) scanner and the Operating System (OS) of an Automated Teller Machine (ATM) have little in common. Furthermore, the goal is not to reproduce the infamous xkcd comic.

To quote a famous saying made popular in the news business:

‘A dog bites a man’ – that’s a story; ‘A man bites a dog’ – that’s a *good* story. [5]

While replacing a legacy system is hardly news, securing a 20-year-old robot using custom-made solutions (instead of simply isolating it), now that is news!

What this master thesis will try to demonstrate is that it is still possible to secure at least some of those systems using the specific example of an AIBO ERS-7 dog robot from Sony which was commercialised in the early 2000s.

By using adaptable solutions that could be ported to other systems with limited on-board processing power, the aim is to demonstrate that just because a legacy system is still used, this does not mean that the only two options available are either to get rid of it (which is not always possible) or to accept the risk.

An important element to emphasise: this only applies where access to the system is possible, as is the case with the robot and its SDK (which will be covered later).

1.1 Context

In order to demonstrate the possibilities of securing legacy systems, the choice fell on the AIBO ERS-7 from Sony, the last generation of robot dog from Sony before development was halted due to budget cuts in its robotic branch in 2005 [6].

The reasons for the choice of this particular robot were multiple:

- A model was available and in working order.

- No prior research focusing on the security of its system or security of its predecessors could be found.
- It had the most powerful hardware out of the series of AIBO before the launch of the ERS-1000 in 2018 [7] [8].
- The development kit for the AIBO has been available since 2002 [9]. This came after Sony faced backlash from the community following a Digital Millennium Copyright Act (DMCA) strike targeted at a programmer who had made available additional tricks for the robot for free [10].
- Programs are stored on the ERS-7 via a Memory Stick, making access and changes easier.

1.1.1 Presentation of Sony's AIBO ERS-7



Figure 1.1: AIBO ERS-7
Credit: Official Sony gallery

The AIBO ERS-7 is the third generation of AIBO robots from Sony and the last before the dissolution of Sony's robotic division (which was revived around 2016 with the ERS-1000, the newest in the line of AIBO).

It runs the proprietary real-time operating system Aperios (as confirmed by the FAQs on Sony's official website regarding the AIBO), the commercial descendant of Sony's Apertos research OS [11], which was deployed in the AIBO line.

It featured several hardware enhancements compared to its predecessors:

- A 64-bit RISC Microprocessor without Interlocked Pipelined Stages (MIPS) processor (RM7000 [8] [12]) clocked at 576 MHz. It was the first MIPS processor to bring the 256 KB secondary cache on-chip [13].
- 64MB of Synchronous Dynamic Random-Access Memory (SDRAM).
- A built-in 802.11b wireless card supporting WEP64 and WEP128 encryption.
- A removable Memory Stick media used to store OPEN-R programs, additional software and media.

- A Lithium-ion battery with a capacity of 2200 mAh which ensured around 1.5 hours of runtime (which varies depending on the battery's age or the time elapsed since the last battery repack).
- A total of 20 degrees of freedom across all limbs (for example, each leg has 3 individual motors).
- A 350 000 pixel Complementary Metal-Oxide-Semiconductor (CMOS) image sensor.
- A battery of sensors (2 infrared distance sensors, an acceleration sensor, a vibration sensor and multiple input sensors).

While official repair support from Sony ended in 2013 for the last batch of ERS-7 (which equipped MIND 3, the final version of AIBO "personality"), an active community revolving around repairs, physical meetups and sharing of information still exists; notable examples include the **RobotCenter** Discord server which is still very active, the `r/AIBO` subreddit, multiple Facebook groups and people offering repair services.

Aside from enthusiasts and collectors, multiple research projects were carried out which included or focused on the ERS-7 model. Reasons for this choice are multiple, the main one being that, at around \$1600 at launch (which is equivalent to \$2800 today, taking inflation into account), it was a highly capable robot at a comparatively affordable price.

The ERS-7 was also used in elderly care [14], hospitals [15] and schools [16], one of the reasons being its appearance which was the closest one to a dog out of the series of AIBO robots (again, before the appearance of the ERS-1000).

The robot had a built-in wireless card which gave it the capacity, among others, to be remotely controlled. The Wi-Fi configuration can either be done using Sony's *WLANManager* software or by directly modifying the file `WLANCONF.TXT` which can be found in the folder `SYSTEM/CONF` of the Memory Stick of the AIBO.

An already important element to note is that, as the wireless card only supports WEP with keys of 64 or 128 bits, when using the remote capabilities of the ERS-7, only two options exist:

1. Using an access point with no encryption enabled, which exposes all data exchanged.
2. Using an access point configured with WEP.

1.2 Problem statement

The problem statement can be summarised as:

"Can the remote network control of a legacy robot be adapted using modern cryptographic techniques (confidentiality, integrity and mutual authentication) using only the platform's original toolchain, and what are the security and performance limits of such a retrofit?"

As it has been pointed out above, the ERS-7 was, and still is as second-hand models can be found on eBay and other online shops for prices ranging from \$800 to \$1800, a sophisticated

quadruped platform that was available to consumers. Even in 2013, Kertész [8] wrote that no equivalent existed.

While modern alternatives like the Mini Pupper 2 built on ROS 2 or the Unitree Go2 Air exist brand-new at a similar price, legacy systems remain heavily used in industrial and healthcare environments.

Examples of those are numerous: a 2021 study by Kaspersky Labs showed that 73% of healthcare providers still used medical equipment with a legacy OS [17] and in a security analysis conducted in 2017 by Trend Micro Inc. in cooperation with researchers from the Polytechnic University of Milan [18], it was demonstrated that among industrial robot users, security risks and safety measures were given little consideration.

Additionally, using an ABB 6-axis IRB140 industrial robot, they demonstrated that several vulnerabilities existed such as missing code signing, poor runtime isolation and naïve cryptography (caused by a design choice focusing on safety rather than security).

To explore this further, it is worth mentioning that this problem also arises in the case of Programmable Logic Controllers (PLC) and related systems (Supervisory Control And Data Acquisition (SCADA), Distributed Control Systems (DCS), power grids and automotive systems).

Those systems are designed to be used over a long period of time:

- The average age of vehicles on the road in the EU is 12.8 years per the ACEA (cars are at 12.3, vans 12.5, trucks 13.9 and buses at 12.5) [19].
- A PLC will last on average 10 to 20 years (though this depends on the environment in which it operates) [20].
- An industrial robot lifetime will be shorter on average, around 6-7 years [21] (the clearance between the mating gears of its joints, caused by wear, is one of the reasons for replacement).
- The case of power grids is somewhat more specific, as they are designed to last even longer (around 30 to 50 years in theory; however, in the case of power transformers, for example, a study found that their life expectancy was actually much shorter, averaging around 20 years [22]).

The fact is that, on one hand, nowadays some of those systems are insecure but still need to be used which leads to a necessity to find methods to secure them; and on the other hand, it is clear that, as technology evolves, it is of paramount importance to find methods to design systems that must stay secure over their entire lifecycle as well as providing them with necessary options so that their security can be improved in the future.

On this second point, it is important to note that there is a real gap in research into ensuring the security of these kinds of systems, built today, throughout their entire lifetime [23].

The ERS-7 is representative of discontinued and no longer maintained embedded devices whose properties (unpatchable link-layer, proprietary OS, no vendor support) mirror those found in industrial and healthcare settings. What is more, considering the fact that the robot was used in those environments, it can be argued that such devices constitute a security risk in those environments. These tend to rely on outdated systems with security not being the primary target.

Remote control of the robot's movements is possible through the Aibo Entertainment Player, Aibnet or Gostai's computing platform URBI [24] (before Gostai's merge with Aldebaran Robotics). Other options exist, like Tekkotsu, but none of them easily allow a user to modify the data link between the robot and a computer.

This leads to the following research questions:

1. Can confidentiality and integrity be provided at application level within the ERS-7's limitations?
2. What is the cost of the retrofit in latency and footprint, and does it remain usable for real-time teleoperation?
3. Can the endpoints be mutually authenticated and what residual risks remain?
4. What is the impact of an attacker gaining physical access in terms of security?
5. Can the resulting secure remote control be interfaced with modern controllers using ROS 2's middleware?

1.3 Structure and contributions

In the following chapters and sections of this document, after having presented a review of the state of the art in research and projects already carried out on and in relation to the AIBO ERS-7, the following methodology will be used:

1. Presentation of an initial prototype of a remote-control console for communicating with the robot.
2. Implementation of a secure version of this console using ChaCha20, Poly1305 and Ed25519.
3. The addition of the ability to remotely control the robot via the console.
4. The use of ROS 2 to interface the robot with modern control systems, such as a computer keyboard.
5. Overview of the testing and measurements applied to the final, secure version of the remote control console.

The following contributions can be presented:

1. First known security analysis of the AIBO ERS-7 robot. Although modifications to its software were available quite soon after its release which implies a certain level of analysis, no known academic work related to the specific aspect of its security was found.
2. First time, unless evidence to the contrary emerges in the future, that a working Authenticated Encryption with Associated Data (AEAD) and Ed25519's signature scheme have been retrofitted within the OPEN-R SDK using its original toolchain.
3. First time that a persistent-counter replay mitigation for this entropy-less device has been implemented.
4. First time that a ROS 2 bridge has been created and tailored for this robot (although there are some traces of projects for ROS1).

5. First thorough assessment of the cost of adding these security measures to this system (proprietary OS and hardware) to be carried out.

Chapter 2

Background and related work

Before delving into the subject of existing research, it is important to provide some context on how OPEN-R works and, consequently, how a program tailored for AIBO works.

2.1 Key features and functionalities of OPEN-R

2.1.1 TinyFTPD

In order to implement a network-capable object, several elements and the program outline were adapted from one of the sample programs available with the OPEN-R SDK: `TinyFTPD`. This is a program that is used to set up a small File Transfer Protocol (FTP) server on AIBO robots with wireless Local Area Network (LAN) capacity. Its main purpose is to enable easy exchange of files for development purposes so as to avoid having to physically remove the Memory Stick every time there is a need to add new files, which also requires switching the robot off and on repeatedly.

An advantage of OPEN-R program development is that as OPEN-R is an Application Programming Interface (API), there is no need to write an entire Transmission Control Protocol (TCP)/Internet Protocol (IP) stack from scratch. Instead, a programmer can rely on reusing the already established Aperios Network Toolkit scaffolding which can be seen in use in `TinyFTPD`.

It should be noted that the Internet Protocol documentation makes multiple references to supposed code examples related to using OPEN-R for network related objectives, similarly to the already available sample programs, but those are nowhere to be seen. Even after extensively searching online, on the Wayback Machine and on community forums, no answers were found regarding the location of these examples. An educated guess, taking into account the way the OPEN-R SDK came to be made public and the fact that page 6 of the documentation indicates that the IPv4 protocol stack is implemented using the OPEN-R Networking Toolkit, is that those examples were made available for Sony developers and researchers working on OPEN-R and were never made public. The reason for this choice, however, is left unanswered.

2.2 Creating a TCP endpoint

Before your object can open a TCP connection, it must create a new TCP endpoint. Your object requires a new endpoint for each TCP connection that it opens. The process for creating an endpoint is the same for each protocol in the IPStack, and is described in detail in "How your object communicates with the protocol stack."

See the `/IPv4_ROOT/Examples/TCP` directory for a full working TCP example.

Figure 2.1: Ghost code samples

2.1.2 OPEN-R objects

OPEN-R (and Aperios by definition, as OPEN-R is only an API) being object-oriented and modular, it makes it easy to individually sketch and implement objects. Those can be later connected. OPEN-R applications are made up of OPEN-R objects which can be understood, as stated in the Programmer's Guide, as processes in Unix or Windows. For example, an application where the goal is to make the robot follow its pink ball will be made up of multiple objects: an image-recognition object using the robot's frontal camera and calculating the ball's position, an object responsible for adjusting the robot's joints, an object responsible for lighting up the robot's interface, etc.

Objects exchange information using a message-passing system where a message is made up of data and of a selector (integer that specifies a task to be done). When an object receives a message, the corresponding function is invoked with the data in the message as its argument (in this case, a function is called a "method"). However, an important limitation is that an object can only process one message at a time; thus, when a new message is received while the processing of the previous message has not finished, the message is queued and processed later. An object cannot terminate itself as it persists as long as the system is activated.

Per the Programmer's Guide, the lifecycle of an object can be visualised like this:

1. Object is loaded by the system.
2. Wait for a message.
3. When a message arrives, it executes the method corresponding to the selector specified in the message. If needed, it will send more messages to other objects.
4. Once the method has finished its execution, it goes back to waiting for a message.

2.1.3 Inter-objects communication

As each object has a specialised, dedicated task like image recognition or network communication, they need to be able to communicate efficiently. To do so, each object defines:

- *Subjects*: the side that sends data using a **NotifyEvent** that includes the data to be sent. If it receives **ASSERT-READY** from the observer, then it starts to send data as the observer is ready. In the opposite scenario, the observer sends **DEASSERT-READY**.
- *Observers*: the side that receives data; it will send a **ReadyEvent** to inform if it is ready to receive data (**ASSERT-READY**) or not (**DEASSERT-READY**).

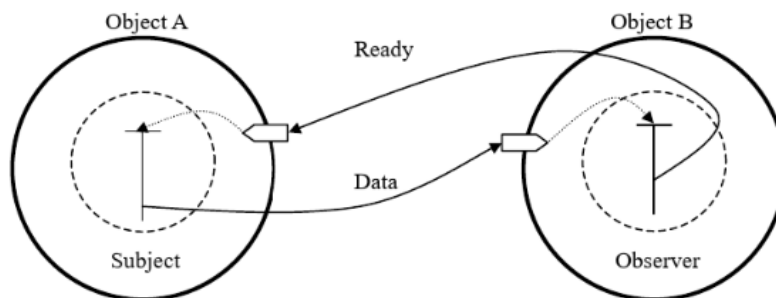


Figure 2.2: Inter-object communication as represented in the Programmer's Guide

In the case of TinyFTPD, there is only one object (TinyFTPD itself) and it defines one subject (TinyFTPD.DummySubject.DoNotConnect.S) and one observer (TinyFTPD.DummyObserver.DoNotConnect.O). As their names suggest, the object does not interact with others but OPEN-R mandates defining at least one dummy subject and observer.

The names specified for the subject/object are called services and they are built following this structure:

1. Object Name: usually the core class name, here **TinyFTPD**
2. Sub-name: the service name which must be unique, here **DummySubject**
3. Data name: must correspond to the data type used in inter-object communication, here **DoNotConnect**
4. Service type: S(ubject) or O(bserver)

2.1.4 Core Class Lifecycle

TinyFTPD is implemented as a core class that inherits from the base `Object` class. In the Programmer's Guide, it is stated that *a core class represents an object which should only be represented by one core class. Objects have entry points for receiving messages and each of those entry points corresponds to a particular method of the object.*

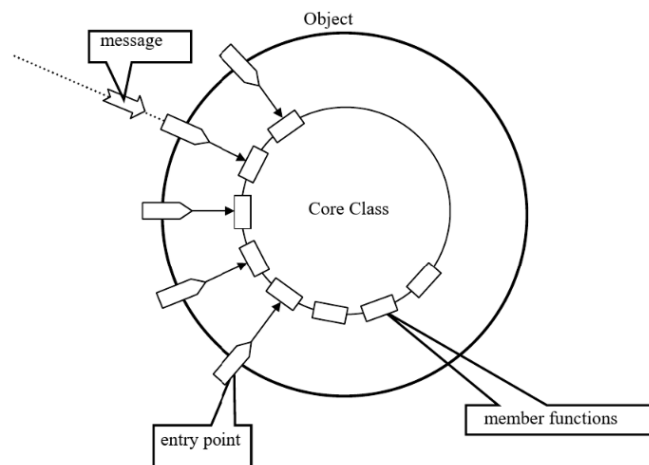


Figure 2.3: Core class as represented in the Programmer's Guide

More importantly, a core class needs to override four specific member functions:

- **DoInit**: called on startup, registers the communication services.
- **DoStart**: after the previous function is executed in all the objects, used to initiate the ANT stack and initialise the connection in a network-capable object.
- **DoStop**: called at shutdown, used to terminate the connection in TinyFTPD.
- **DoDestroy**: called after **DoStop**, can be used to delete all observers and subjects generated in **DoInit**. When the object does not interact with other objects, the function is left empty.

To help define these functions, OPEN-R offers predefined macros from `<OPENR/core_macro.h>`.

2.1.5 ANT Messaging

As the project needs to use the robot's IP stack via shared memory buffers (rather than local stack variables), several network-related elements of TinyFTPD are reused which interact with the IPStack (OPEN-R system layer object). Shared memory buffers are needed as objects need to send requests for network services to endpoints in the IPStack (similarly to how objects communicate to each other, the difference being that special messages are used for these exchanges):

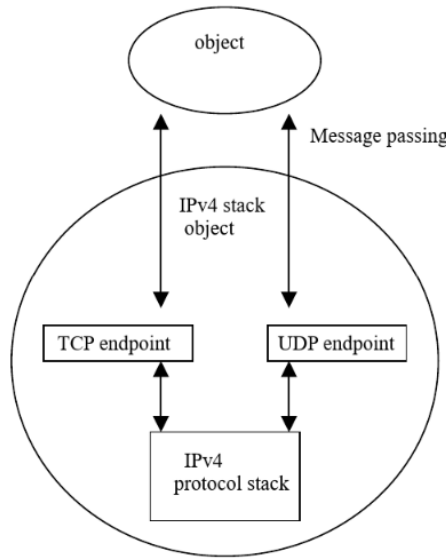


Figure 2.4: Object communication with IPStack as represented in the Programmer's Guide

`antEnvCreateSharedBufferMsg` is used to allocate memory for incoming and outgoing data, mapping them to `sendData` and `recvData` pointers.

The TCP protocol is used in TinyFTPD and will also be used in the network-capable object as TCP is reliable and connection-oriented. There is no need to stream images from the camera or sound from the microphones making the use of User Datagram Protocol (UDP) unnecessary. On OPEN-R, there are 5 available operations to objects for TCP:

- Connect: open a TCP connection to an object on another host.
- Listen: start listening for connection requests.
- Send: send data over an open connection.
- Receive: receive data from an open connection.
- Close: close a TCP connection.

Those operations are performed by sending special messages to a TCP endpoint (for example, to close, `TCPEndpointCloseMsg` is used).

2.1.6 stub.cfg

The `stub.cfg` file is used to connect an object's entry points to the member functions of its core class and in the case of network events, **Extra** ANT entry points are defined in addition to the services.

2.1.7 Stubgen2

The `stub.cfg` file is read and used by the `stubgen2` binary to generate intermediate files to connect methods of an object and the member functions of a core class. While those files are not necessarily important to fully understand, any error in the "*source*" files, which may not be detected by the compiler, will have an impact on these generated files and cause a crash.

2.1.8 Object memory allocation

A distinctive feature of an OPEN-R program is that the allocated memory must be specified at design time using a file with an `.ocf` extension.

The parameters are:

1. The name of the object: `tinyFTPD`
2. The size of the stack in bytes which is not extended at runtime: `3072`
3. The number of bytes used to extend the heap space (as extending the heap is a costly operation, a value too small could lead to frequent heap resizing): `16384`
4. Scheduling priority is represented by an eight-bit **unsigned** integer where the upper four bits specify the scheduling class (an object with a lower class will not run if an object of a higher class is running) and the lower four bits control the ratio of execution time between objects within the same scheduling class (higher value equals more time for the object): `128` (recommended scheduling class for an ordinary application)
5. Cache memory enabled or not (`cache` or `nocache`): `cache`
6. Memory address space allocation (`tlb` for virtual space or `notlb` for physical space), the value is ignored with the `nomemprot` configuration: `tlb` (recommended and should be used with `user` mode)
7. Either `kernel` mode (used with the `nomemprot` configuration) or `user` mode: `user`

2.1.9 Makefile and linking

Makefiles are used to call the `mipsel-linux-g++` compiler with appropriate flags, the `stubgen2` binary, necessary libraries and define available Make commands (`make` to compile the binary, `make install` to move the binary into the `MW/OBJS` folder and `make clean` to remove files generated by the compilation).

The Makefile used for the project is very similar to the one used in TinyFTPD and can be found in Appendix 1.

Another important limitation of Aperios is that it follows a legacy filename format named 8.3 where the base name must have a maximum of eight characters and the extension

must be three characters or less. Both must be separated by "." and lowercase letters are automatically converted to uppercase letters. Digits zero to nine and these specific characters are supported: `$&#{}~%'()-@^!_.`

This limitation being only mentioned once in the appendix of the Programmer's Guide, it can be easy to get it wrong and the corresponding error message will be:

```
1 Downloader::Download cannot open module file : /MS/OPEN-R/MW/OBJS/<FILE>
2 Downloader::deleteRegion cannot delete region (status=2).
3 Downloader::deleteRegion cannot delete region (status=2).
4 [oid:80000033,prio:1] OObjectRegistry::LoadObjects() : Load(/MS/OPEN-R/MW/
  OBJS/<FILE>) error 90
```

2.1.10 Debugging

There exist three ways to debug an OPEN-R program:

1. When the compiler encounters a syntax error, an error message will be displayed and the binary will fail to compile.
2. For certain types of errors, such as a file error, the AIBO wireless console will display an error message (only if the configuration `WCONSOLE` is used, otherwise the console is not available). If used in the code, `OSYSDEBUG` and `OSYSPRINT` messages will also be displayed. They should be used frequently as those are the primary and most straightforward way to debug.
3. In case of a Central Processing Unit (CPU) exception (for example invalid address reference, instruction execution or memory access), the robot will emit a special sound and the Exception Monitor will write a file in `OPEN-R/EMON.LOG`. The Exception Monitor behaviour can be adjusted in `OPEN-R/SYSTEM/CONF/EMON.CFG`. However, the process of debugging a CPU exception requires a thorough knowledge and understanding of how the MIPS architecture works as it will involve reverse-engineering the runtime memory state back to the static binary at the origin of the exception. The Programmer's Guide gives a structure in order to facilitate things but the process stays complex. No CPU exceptions were encountered throughout the project.

2.2 Related work and literature

The ERS-7 has been the subject of a significant body of research, which can be split into different categories:

- Simulation [25] [26]
- Movements [27] [28]
- Surface recognition [29]
- Sensors usage for localisation [30]
- And other various topics related to the RoboCup competition [31] [32] [33] [34] [35]

While that research is highly interesting in its own right, it does not focus on the issue of wireless communication security for the robot.

The only research focusing on the ERS-7 that is relevant to the work carried out comes from Kertész from the Tampere University of Applied Sciences (Finland) [8]. The research focuses on updating and enhancing the OPEN-R environment in order to facilitate future development and achieve better performance.

This took the form of using additional, specific compiler flags to reduce the size of the final binaries produced when compiling OPEN-R projects, the implementation of an algorithm to enhance the stability of the ERS-7 wireless connection when used with UDP and updating the toolchain in order to make it compatible with the latest (at the time) gcc versions (4.4-4.6.1).

These enhancements were used in the AIBO+ open-source project which had been developed by the same researcher. The impact of the additional compiler flags resulted in binary sizes reduced by 24.06%, the network algorithm ensured that the connection could be recovered in most situations and modern programs can be implemented more easily for AIBO thanks to the updated version of the toolchain.

The key takeaway from this research, with regard to setting up a secure network connection on the robot, is mainly the update to the toolchain and the compiler. The problem is, apart from the fact that the updated version is now 10 years old, that there is a risk that changes made to the toolchain may cause compatibility errors when used with the robot. The option to use this toolchain in the future, once the secure remote connection platform is functional, can be retained.

2.2.1 Securing legacy robots

Research on the subject of securing legacy robots is not extensive, the main reason being surely that the first mass-produced commercialised robots only appeared 20 years ago, but this issue is set to become increasingly important with the boom in the adoption of domestic robots, alongside that of AI.

Nevertheless, a study focusing solely on extending the service life of a legacy robot is worth mentioning [36]. This research was conducted by members of the U.S. Naval Research Laboratory’s Laboratory for Autonomous Systems Research (NRL-LASR) and focused on the Xitome Mobile Dexterous Social (MDS) robot which was released in 2008.

While this research did not put its main focus on security but rather on implementing a lightweight middleware acting as an intermediary between legacy motion control systems and modern tools such as Robot Operating System (ROS), this middleware offers a supposedly secure SSH tunnel ensuring that state and reference data can be exchanged between the main and user controller. The study does not go into detail about the implementation of SSH or which algorithms are enabled, an important point as some implementations of SSH, such as OpenSSH, still allow deprecated cryptographic algorithms.

It can be considered another attempt to develop a robot’s architecture standard, here by following the *x-Ach* architecture and applying it to the MDS robot. The authors acknowledge that other robotic frameworks already exist but point out that this specific architecture makes the most of their advantages whilst leaving their disadvantages aside. Despite all this, it seems that the architecture was only applied to the MDS and Hubo.

Security risks in modern robots

To broaden the scope a little, it is worth noting that as the field of robotics has been evolving continuously at an almost exponential rate over the last 20 years, the use of the term ‘legacy’ when referring to modern robots can be broad. While there is no discussion about the fact that the MDS or AIBO ERS-7 can be considered legacy, robots such as Aldebaran Robotics’ Pepper, which had been available for sale between 2014 to 2020, are in a sort of a grey area. It was still on sale six years ago, which makes it recent enough not to be considered legacy, but it was first made available 12 years ago.

This robot was the subject of a security assessment in 2018 which highlighted multiple security flaws. The interesting aspect of this research is that the authors were already highlighting the risks coming from an *"overlook of a number of well-established security best practices and countermeasures that resulted in exposing the final product to basic, yet very dangerous, security flaws which could be easily preventable"*. They also emphasised the feeling that *"commercial robots get on the market too quickly, evolving from research frameworks to final products without enough prior security investigation."* [37]

It is noteworthy that in 2017, one year before the publication of this research, \$2.7 billion was raised for the robotics industry [38] while \$18.8 billion was raised mid-2026 [39].

If the trend of neglecting security continues despite the surge in investment, this will only serve to increase security risks and incidents.

2.2.2 Security strategies on legacy systems

One might ask why not simply isolate those robots or systems in an environment cut off from the rest of the network. In the case of the ERS-7, remote control could be carried out over an isolated wireless LAN, for example.

The subject of securing systems that are expected to have a long lifetime (often exceeding 10 years) has already been discussed in Section 1.2.

However, a major problem affecting robotics and autonomous systems [40], as well as PLCs, SCADA systems and power grids [23], is that the current security approach related to cyber threats and other breach of sensitive systems has been largely responsive. Security risks tend to be addressed only after the system has been put into operation or has become too old.

Consequently, measures are being put in place such as:

- Air-gapping
- Program detection and verification
- Firmware audit
- Side-channel detection
- Intrusion detection
- Honeypot-based detection
- Digital forensics (in order to trace the source of a security incident)

Air gapping is the simplest solution to implement in the case of an isolated system. However, the biggest flaw of air gapping is that it can be violated through a variety of vectors such as physical vectors [41], insider threats (rogue employee), covert channels or supply chain attacks.

What is more, with the arrival of Industry 4.0, systems are becoming increasingly interconnected (and connected to the outside world). In the case of newer systems, an air gap is simply not a viable solution.

As mentioned above, the measures put in place are merely reactive rather than proactive, and they too have their limitations [23]:

- Program detection and verification can be heavily challenged by false positives/negatives, dynamic environments that constantly change, encrypted code that cannot be analysed, complex network architecture or limited understanding of the overall context of the process.
- Firmware detection also faces a major obstacle consisting of a lack of databases containing a sufficient number of signatures (further complicated by the lack of standardisation) which leads to poor visibility. They also cannot protect against zero-day exploits.
- Insider threats, integration difficulties, encrypted traffic and the challenge of detecting violations of security policies all seriously reduce the effectiveness of intrusion detection.
- Attempts to detect side-channel attacks are limited by an increased sensitivity to physical modifications, a limited scope, and vulnerabilities to advanced attacks that circumvent newer detection systems.
- Similarly, the use of honeypots and digital forensics also faces recurring challenges due to the nature of the targeted environment.

New possibilities that would allow for better parallel development alongside ageing systems or use with legacy systems are possible:

- Increasing the usage of secure embedded systems such as embedded hypervisors, CPUs with security processors, frameworks for secure firmware updates or secure boot mechanisms.
- Using secure communication protocols instead of updating protocols that cannot be made secure by their very design (even newer versions are still riddled with vulnerabilities [23]).
- Virtualising PLC-based systems.
- Using open-source units to make possible the creation of testing environments. This would significantly improve research focusing on the security of these systems and make it easier to find better solutions in situations where, at present, physical access to the systems under study is often required (such as in [18]).
- Making use of cloud-based computing.

In the specific case of the ERS-7, an additional secure embedded system fitted onto the robot (like an embedded Wi-Fi module with cryptographic coprocessors) or virtualising

some of the AIBO's systems would not be architecturally feasible. There is no wired segment on which to append a device or, at least, without making permanent, irreversible and potentially destructive changes to the AIBO. Similarly, virtualising some of the systems would not solve the security problem posed by the network link as the robot's endpoint is its built-in 802.11b radio, virtualised systems would still have to pass through it which would remove any additional security at network level.

A gateway would still leave the network link between the robot and the access point insecure and an attacker within radio range would bypass the isolation perimeter.

In order to ensure confidentiality of exchanged data through the network link, two main options exist: symmetric and asymmetric encryption.

Asymmetric encryption is much more computationally intensive and is significantly slower than symmetric algorithms [42]. Modern usage involves a trade-off where asymmetric cryptography is used to transmit securely a secret key which is then used to encrypt the following exchange of data using a symmetric encryption algorithm.

The Transport Layer Security protocol, specified for version 1.3 by RFC 9846, is built on this principle using an asymmetric cipher to establish a session-specific key which is then used to encrypt further communication using a symmetric cipher.

There exists a wide range of symmetric algorithms, the Ascon family being one of them. They are authenticated ciphers which were selected in February 2023 by the U.S. National Institute of Standards and Technology as the standard for cryptography on constrained devices, and more specifically its authenticated encryption and message authentication. Furthermore, a C implementation is available on GitHub in multiple flavours depending on the architecture (Advanced RISC Machine (ARM), ESP32, x86-64) and was designed by the Ascon's team as a reference implementation.

On the other hand, a more conventional combination of algorithms used in symmetric cryptography is ChaCha20+Poly1305. ChaCha20 is a stream cipher that only relies on addition, exclusive-or and rotation operations which *are uniformly fast across most CPUs* [43]. As a result, it is much faster than Advanced Encryption Standard (AES) with the exception of CPUs that have the AES-NI instruction set extension (which is not the case with the ERS-7's processor). Poly1305, on the other hand, which is a Message Authentication Code (MAC), was originally designed for use with AES but the most common combination is ChaCha20+Poly1305 and is supported by a wide range of protocols (IPsec, Transport Layer Security (TLS) v1.3 and OpenSSH since version 6.5, to name a few). Poly1305 was designed to be of very consistent high speed, to have low per-message overhead and guaranteed security **if** the scheme used in conjunction is secure [44].

As a result, there is a much wider choice of implementations in either C or C++ and this makes it possible to choose an encryption method that has already been widely tried and tested compared with more experimental and specialised ciphers.

Chapter 3

Practical work and implementation

This chapter presents the practical contribution of this work which consists of the design, implementation and integration of a secure remote-control communication for the AIBO ERS-7. The system was built entirely using the original toolchain.

The objective is to establish confidentiality, integrity and mutual authentication over a network communication whose link-layer protection cannot be modified.

The sections follow the order in which the work was carried out, and each one constrains the next. The first section examines the security of the built-in wireless link and of the debug console, and establishes what cannot be changed on this platform.

Section 3.2 describes the development environment that had to be reconstructed before any code could be run on the robot, including the creation of a Programmable Memory Stick (PMS).

Sections 3.3 and 3.4 then identify the different threats by applying the STRIDE methodology and then prioritising them by urgency and impact; only the high-priority threats are addressed by the implementation.

Section 3.5 presents the OPEN-R program (TinyConsole) developed incrementally:

1. An unauthenticated prototype establishes the transport using a naive XOR cipher.
2. A ChaCha20-Poly1305 layer adds confidentiality and integrity.
3. An Ed25519 handshake adds mutual authentication.
4. A persistent counter adds replay resistance.

Finally, Section 3.6 makes use of the resulting channel through ROS 2, enabling a twenty-year-old robot to be driven by modern robotics middleware without weakening the security guarantees established beforehand.

The elements implemented here – the OPEN-R program, the Python client and the ROS 2 bridge – are the subject of the security testing and performance measurements reported in Chapter 4.

3.1 WEP security – or rather, the lack of it

As was detailed in the presentation of the ERS-7, the built-in wireless card only supports the Wired Equivalent Privacy encryption protocol. While WEP was known to be insecure as far back as 2001 because of several vulnerabilities in the key scheduling algorithm of RC4 [45], there were no widely available alternatives until 2003 and 2004 when the Wi-Fi Protected Access (WPA) security protocol, and its successor WPA2, became available.

It was possible to implement WPA through firmware upgrades on wireless network interface cards but Sony never proposed any update to the AIBO wireless card. Multiple reasons

could explain this absence of reaction like the fact that as Aperios was not based on Unix and was custom-made by Sony’s engineers, they would have had to implement a solution from scratch to support WPA.

As the Aperios OS is not available to researchers or developers working on the AIBO, there are no known ways to make the wireless card compatible with WPA or WPA2.

Tools designed to break WEP security like Aircrack-ng became widely available as early as August 2001 (version 0.0.9). Aircrack-ng passively monitors encrypted transmissions on a WEP network and computes the encryption key when enough packets with weak IVs have been gathered.

In order to demonstrate those statements, attempts were made to set up a wireless network encrypted with WEP using a TP-Link Archer A5 V4 router running the Linux-based firmware OpenWrt. Those attempts failed for multiple reasons and no WEP-cracking was performed.

First of all, the latest available version (25.12) has been stripped of any support for WEP. Secondly, as modern operating systems and networking stacks (such as Windows since Windows 10’s 1903 update and contemporary Linux distributions) have fully deprecated WEP support, testing the legacy encryption in real-world conditions with a functioning ERS-7 proved challenging.

Finally, as the vulnerabilities of WEP have already been widely demonstrated on the strength of [45] and the tooling history, it was decided to bypass link-layer security entirely and to focus the work on the ERS-7 and its wireless communication at the application layer.

3.1.1 Telnet console

It must be noted that the OPEN-R SDK provides 3 configurations when compiling and running programs (per the official OPEN-R SDK Programmer’s Guide):

- BASIC: without a wireless LAN environment
- WLAN: with a wireless LAN environment and without a wireless console
- WCONSOLE: with both a wireless LAN environment and a wireless console

The WCONSOLE mode gives access to a `telnet` console which is mainly used to display debug and error messages.

As `telnet` does not encrypt data and the console lacks any type of authentication, it is considered a security vulnerability and must be deactivated when pushed into production as it gives insight into the current execution of the robot. Still, it can be very useful during development.

3.2 Setup and necessary assets

3.2.1 OPEN-R SDK

The OPEN-R SDK, per the official Installation Guide from Sony, was certified to run on Windows XP (via Cygwin) or any operating system supporting:

- Standard UNIX commands such as `cp`, `perl` and `sh`.
- GNU `make`
- GNU `binutils`, `gcc` and `newlib` (built for a MIPS cross-development environment).

The necessary hardware involves a CPU clocked at 233 MHz or above, 64MB of RAM and more than 200MB of free space in addition to a memory stick reader/writer.

The source files for the SDK (excluding those required for development via Cygwin) consist of:

- OPEN-R SDK v1.1.5: the SDK, several sample programs for the ERS-200, ERS-7, common programs and official manuals.
- `binutils` v2.14: the 2003 version of GNU `binutils` which are a collection of library tools including, for example, `ld` (GNU linker), `as` (GNU assembler) and `strings`.
- `gcc` v3.3.2: the October 2003 version of the GNU Compiler Collection
- `newlib` v1.10: the February 2002 version of a C library intended for use on embedded systems which is a conglomerate of several library parts (per the Sourceware website).

While the list of tools is only the minimum set necessary for using the SDK, updating them would risk compatibility problems with the SDK.

As using Windows would have created more problems and solved none (the necessary version of Cygwin would have needed to be from around the same time period), it was decided to use an Ubuntu VM in VMWare for development purposes.

Multiple attempts were necessary, first using Ubuntu 18.04 LTS (2018), then 12.04 LTS (2012) and finally 8.04 LTS (2008) which was the most recent stable release of Ubuntu found to be able to compile the old SDK. The reason for this iterative process was to try to find a release old enough to work with the SDK but recent enough to be able to make the most of the tools and user-experience offered by the latest versions of Ubuntu.

It was necessary to install those additional packages once the Virtual Machine (VM) was set up:

- `flex`
- `gettext`
- `patch`
- `bison`
- `texinfo`
- `make`
- `g++`

As Ubuntu 8.04 could not, and should not, connect to the Internet, the following development process was used when working on the OPEN-R program:

- Write code using the Geany Integrated Development Environment (IDE) which was supported by the OS.

- Compile it via the SDK using `make` and then `make install` inside the program's folder. The first one builds the necessary binary executable, the second moves the binary into the `MS/OPEN-R/MW/OBJS` folder of the program's private copy of the OPEN-R file system.
- Assemble the complete OPEN-R program by copying the files from the memprot WLAN core version of the OPEN-R configuration. A nomemprot (no memory protection) version exists but makes all objects run in kernel mode and risk memory corruption.
- Once the full OPEN-R configuration is assembled, it is transferred to the Memory Stick using a Sony Memory Stick reader (model MSAC-USM1). While third-party readers could work, the online community recommends using one from Sony to avoid problems.
- When needing to transfer files from the guest OS to the host OS (Windows 11), the solution used was to start a local HTTP server with Python 2.5 (`python -m SimpleHTTPServer`). The reason is that VMWare tools is not compatible with Ubuntu 8.04 which makes it challenging to transfer files easily.

While those limitations could have been alleviated as at least one project to update the SDK libraries and further optimise the compiler had been carried out [8], those modifications were integrated into the AIBO+ open-source project with the goal to implement a new AIBOware software. Using this project as a basis to implement a secure communication could have been more complicated than simply starting from scratch using the samples provided by Sony as a learning aid. It is also worth noting that it would have been necessary to remove all the unnecessary parts and modules of AIBO+ and risk breaking dependencies.

This constraint also frames the work as an exercise in securing the ERS-7 using the original toolchain without modifying it.

3.2.2 Programmable Memory Stick

It was mentioned that the last version of the ERS-7 "personality" was the MIND 3. This version came in the form of a Memory Stick card with necessary files already included.

However, a particularity of those Memory Sticks was that they did not allow users to write custom programs and modifications on them. The reason was that Sony designed AIBO so that it would only accept a specific type of Memory Stick that could run programs built with the OPEN-R SDK and other custom-software: Programmable Memory Stick, which were sold as a pink version of standard-issue Sony's MagicGate (Sony's copy-protection technology) Memory Stick.

What made them different from regular Memory Sticks lies in the fact that they embedded a decryption key located in the first two pages of the second physical block of memory (1KB of data).

True PMS are hard to come by and sell for prices as high as \$100. Fortunately, a workaround exists: using a Sony CLIE Personal Digital Assistant (PDA) with the StikZap software (developed by user AiboPet and available on his website AiboHack), it is possible to transform a generic Memory Stick into a PMS.

The reason why a Sony PDA must be used, instead of another old PDA is that the CLIÉ memory stick driver loads the stick into the memory of the PDA and the program is able to manipulate the logical to physical mapping table (source: StikZap open-source code).

The original source code documentation explains this hardware limitation:

Other devices have the memory stick driver packaged in the stick reader or adapter. The memory stick protocol and data tables are implemented in small embedded processors and custom chips on the device itself. After that PCs or other PDAs see a memory stick as a logical device, packaged as a generic File Allocation Table (FAT) volume. The physical level is not directly accessible.

As PCs and other PDAs use Logical Block Addressing, they abstract the storage which hides system blocks (block 0 contains low-level structure of the memory stick, block 1 is not used and block 2 contains the decryption key information) and only exposes the user-data partition (which begins at block 3).

The way StikZap succeeds in modifying block 2 content is by executing a memory pointer manipulation and redirection within the live system memory even though PalmOS (the OS used by Sony CLIÉ) still enforces safety boundaries. The software also needs to circumvent the way the OS implements the table linking logical and physical blocks.

The CLIÉ used to make the necessary modification and turn a generic Memory Stick into a PMS was the model PEG-TJ25, which uses a Motorola i.MX1 ARM based CPU:



Figure 3.1: Sony CLIÉ PEG-TJ25
Credit: Product page on the Fnac website

StikZap works as follows:

1. It queries the OS using **FtrGet** calls to identify the system's CPU type. In the case of the PEG-TJ25, the architecture is ARM, which sets a positive offset value for **g_magicIndex** (0x1C4/4 for this model of PDA).
2. As the program needs to bypass the OS blocks which prevent direct access to the needed sectors, it must find the memory addresses of the Memory Stick driver. Using **GetPtrsArm** which invokes **CallArmlet**, it then extracts a compiled native ARM binary from the application resources and uses the official API **PceNativeCall** to cross the boundary from the 68K emulator environment into native ARM execution. Inside **armlet.cpp**, the assembly function **_GetPtr_MSLIB_OS5** loads the CPU **r9** register and offsets 896 bytes to get the internal base memory pointer of Sony's **MSLIB** (Memory Stick Library).

3. The base address of the driver is then taken and the `g_magicIndex` variable is used to locate the driver's internal variables:
 - `pwTable`: live translation array mapping the Logical Block Addresses to physical blocks
 - `pbTwoBitUse`: block state table tracking whether physical sectors are free, in use or reserved
4. Once the "Zap PMS" option is chosen by the user, the function `PerformWriteVerbose` is used which loads the 1024-byte signature array `g_type_PMS`. In order to be able to write into the locked physical block #2, StikZap swaps the memory translations:
 - (a) The original mapping of block 0 is saved.
 - (b) The logical block 0 is pointed to physical block 2.
 - (c) The physical block 2 is marked as an active and writeable block.
5. A single write request might not land predictably as flash controllers automatically rotate physical sectors when data is modified in order to extend hardware lifespan. It calls the `SlotCardSectorWrite` function to target the **logical** block 0 which is interpreted as an authorised call.
6. Once the injection is successful, the program cleans up in order to prevent future errors related to the modified blocks alignment.

3.3 Threat modelling

In order to accurately target the main risks associated with the operation of an ERS-7 via remote control, the STRIDE approach is applied based on the following components as illustrated in the following figure:

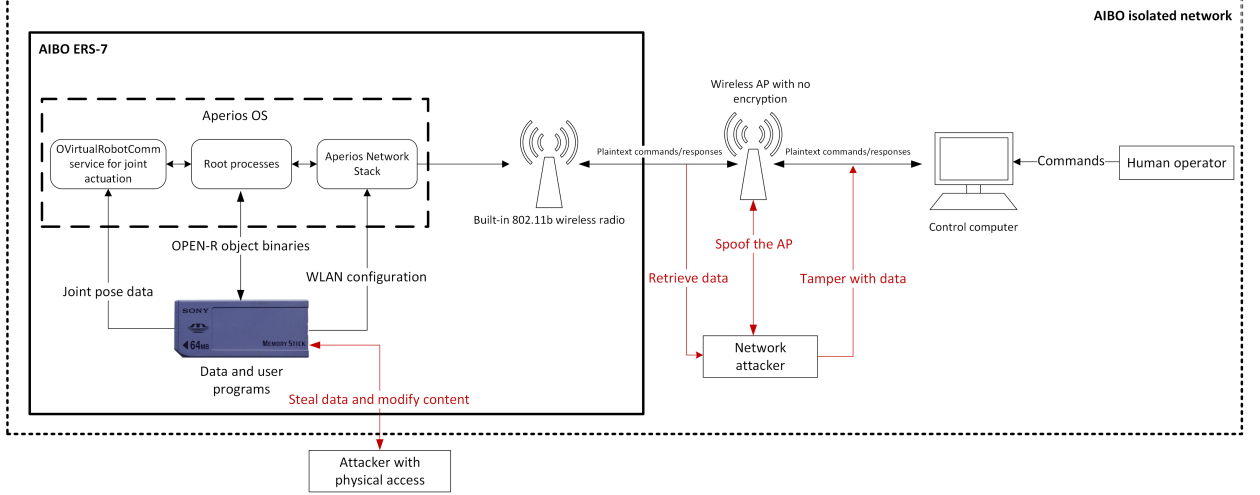


Figure 3.2: Initial AIBO data flow diagram

- AIBO ERS-7 composed of a memory stick (contains a program which enables remote control), a 802.11b built-in wireless radio and its Aperios OS which runs root processes.
- Generic access point that supports 802.11b with a 2.4 GHz channel with no encryption enabled.
- Remote computer running a local version of the program that enables remote control.
- A human operator who uses the control computer.
- A network attacker which is in the radio range of the isolated network.
- A physical attacker which has access to the robot's Memory Stick.

As the data flow between the robot and the computer will not be encrypted at the link layer (WEP no longer being supported and no other link-layer encryption being available), it is considered that the environment of execution is isolated from other networks.

However, as is depicted on the diagram, an attacker within radio range of the isolated network could still retrieve data from the unencrypted communication, spoof the Access Point (AP) or tamper with exchanged data.

Similarly, as the Memory Stick is easily accessible and no encryption nor protection exist on it, an attacker with physical access to the robot could steal or modify data stored on it.

The STRIDE methodology is then applied on the basis of this data flow:

/	S	T	R	I	D	E
External entity	X					
Process		X				
Data Flow	X	X	X	X	X	
Data Store		X		X		

Table 3.1: STRIDE per element

3.3.1 External entities

There is only one external entity:

- Human controller responsible for remotely operating the robot.

The two attackers depicted in Figure 3.2 are not modelled as external entities as they do not represent legitimate actors. Their capabilities are reflected in the threats against the data flows, the OPEN-R program and the Memory Stick.

Threat E1: Spoofing the remote controller

If there is no authentication of any sort, a malicious actor could spoof the control computer either by disconnecting the controller from the robot via TCP RST packet or by connecting to the robot when no other connection exists.

3.3.2 Process

The following are considered processes:

- The OPEN-R program running on the AIBO ERS-7 and stored in the Memory Stick.
- The program running on the control computer.

Threat P1: Tampering with the controller's program

If an attacker gets access to the control computer, they could tamper with the functioning of the program and modify it.

Threat P2: Tampering with the OPEN-R program

If an attacker is able to physically access the ERS-7, they could alter the joint articulation values associated with specific movement commands, hence forcing the robot to execute hazardous actions upon receiving otherwise benign remote instructions.

3.3.3 Data Flow

The following are considered data flows:

- The connection between the robot and the access point.
- The connection between the computer and the access point.

Threat DF1: Spoofing the access point

An attacker could spoof the access point in order to intercept credentials or to send incorrect commands.

Threat DF2: Tampering with the wireless connection

A malicious actor directly tapping into the wireless connection could modify packets before they reach either the robot or the controller's computer in order to falsify exchanged data.

Threat DF3: Issuing malicious commands

In the absence of stored logs on either the robot or the computer, any malicious data modification causing instability or malicious actions could not be traced back to its origin. This would lead to a situation where actions taken by either an attacker or the remote controller could not be assigned and verified.

Threat DF4: Exposure of exchanged data

In the case of a Man-in-the-middle attack, an attacker could capture exchanged data, store it and decipher them if they are exchanged in plaintext or if the encryption used is not strong enough.

Threat DF5: Denial of Service

If an attacker executes a TCP RST or FIN flood attack, it could cause the AIBO CPU to get overwhelmed and crash. It would also be possible to cause the connection between the robot and the controller to drop if there are no mechanisms in place to prevent an unexpected TCP RST packet from being processed.

3.3.4 Data Store

The following are considered data stores:

- The AIBO's Memory Stick which stores program binaries and configuration data.
- The controller's memory which stores the controlling program and, above all, the cryptographic decryption key.

Threat DS1: Tampering with the Memory Stick

In the scenario where an attacker is capable of physically accessing the robot's Memory Stick and modifying the stored binaries in order to replace or add their own cryptographic keys, they would then be able to remotely connect to the robot and assume control.

The reason why a physical access would be necessary, in order to modify the data stored, is that the Memory Stick features a physical "lock" switch that can be used to enable write-protect mode. Once the switch is flipped, no data can be written to the Memory Stick (mainly used to prevent data loss).

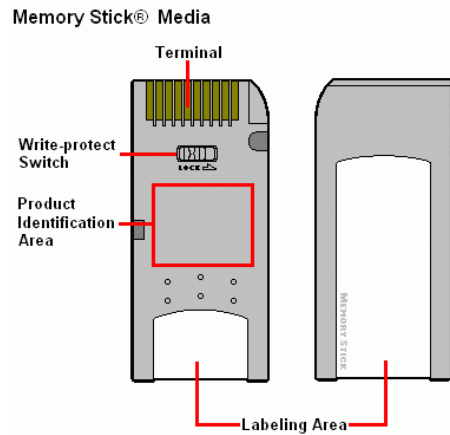


Figure 3.3: Memory Stick Media
Credit: Sony community website

Threat DS2: Disclosure of the cryptographic keys from the controller's computer

In the event of a data breach where the computer memory was accessed by an attacker, the cryptographic keys necessary for connecting to the robot could be retrieved and used for malicious attempts.

Threat DS3: Disclosure of the cryptographic keys from the robot's Memory Stick

In a situation where a malicious actor was able to access the stored binaries in the robot's Memory Stick, they could be able to extract the cryptographic material needed to establish a secure session with the remote computer.

3.4 Risk analysis

In this section, three types of threats are distinguished:

- Threats that require immediate action: they could cause significant damage to the robot or have a very high probability of leaking information.
- Threats that require planned action: they pose a risk but to a manageable extent, are less likely to happen or would require more resources and knowledge to accomplish.
- Threats that can be ignored or solved when more resources are available and previous threats are treated: these risks are either unlikely to occur or their impact would be more limited.

3.4.1 High-priority threats (immediate mitigation)

- E1: spoofing the remote controller
- DF1: spoofing the access point
- DF2: tampering with the wireless connection
- DF4: exposure of exchanged data

The most urgent threats to manage are the ones related to the insecure network transmission and access point. Those are the easiest for a malicious actor to act upon as they do not require physical access and can be done remotely.

Depending on the emitting distance of the wireless access point and its location (indoor or outdoor), this could mean that an actor would only need to be in the vicinity in order to make any of those situations possible.

Those threats will be mitigated by the implementation of an application-layer encryption via a custom-made OPEN-R program, which will be described in the following section.

3.4.2 Medium-priority threats (planned mitigation)

- DF3: issuing malicious commands
- DF5: Denial of Service

For DF3, the risk lies in the fact that the lack of auditing prevents the attribution of actions. However, due to the hardware constraints of the robot, implementing a robust cryptographic logging mechanism would be a significant drain on those limited resources. It would require careful RAM allocation and storage management. In addition to the ERS-7 limited processing capabilities, the physical Memory Stick is restricted to 128 MB of internal storage, making persistent local auditing unfeasible in a continuous-use environment where logs are not regularly removed from the storage medium to free up space.

For the second threat, while a whitelist or ban of MAC addresses could be enabled on the access point, this type of protection would provide only limited protection as any attacker capable of a TCP flood can also trivially spoof MAC addresses. Furthermore, as the wireless network is not connected to any external networks, this would limit the potential for a Denial of Service (DoS) attack as it would require close physical proximity.

An interim solution could potentially be implemented to ignore any packet not originating from the control computer, but this is outside the scope of this work.

3.4.3 Low-priority and accepted risks

- P1: tampering with the controller's program
- P2: tampering with the OPEN-R program
- DS1: tampering with the memory stick
- DS2: disclosure of the cryptographic keys from the controller's computer
- DS3: disclosure of the cryptographic keys from the robot's Memory Stick

Those threats can be divided into two categories:

- Threats requiring physical access to the controller computer's storage
- Threats requiring physical access to the robot's Memory Stick

For the first category, whether or not it is possible to access the computer's storage depends more on the implementation of the operating system and the security measures in place than on the security of the program itself.

For the second category, apart from the physical lock switch, Memory Sticks do not have any encryption capacity for the stored data. However, while it is possible to have the cryptographic keys written as plaintext in the files of an OPEN-R program, it is more challenging to get a hold of them once the program has been compiled.

OPEN-R binaries are compiled using a MIPS cross-compiler (`mipsel-linux-gcc`) which relies on standard `gcc`. The difficulty in reverse engineering would mainly lie in the fact that the binaries are optimised for Aperios where the control flow relies not on direct function calls, but on isolated objects communicating via an internal messaging system. Additionally, internal OPEN-R tools generate C++ wrappers before compiling the code which adds even more bloat to the binary.

In the end, while those elements increase the effort required by static analysis, it is merely a case of security by obscurity. Hardcoding keys on the Memory Stick remains the only option for now as there is no secure enclave or Trusted Execution Environment available.

3.4.4 Summary using a risk scoring matrix

The identified threats have been evaluated and plotted on a standard risk scoring matrix, illustrated here:

		impact				
		trivial	minor	moderate	major	extreme
probability	rare	low	low	P1 & P2 & DS1	medium	DS2
	unlikely	low	low	DF3 & DF5	medium	medium
	moderate	low	medium	medium	medium	E1 & DF1
	likely	medium	medium	medium	DF2 & DF4	high
	very likely	medium	medium	high	high	high

Figure 3.4: Risk scoring matrix (adapted from Mees [3])

3.5 TinyConsole

3.5.1 Reasoning and objective

In order to solve the insecure communication problem, the following development process was defined:

- Build a first prototype of an OPEN-R program that enables the robot to open a TCP port, respond to received commands, and use XOR encryption. At the same time, implement a simple Python script to use from the remote controller regardless of the operating system used.
- Find a symmetric encryption implementation that is secure, fast, requires no hardware acceleration, and can be implemented in C or C++ without using external libraries.
- Adapt the implementation in order to replace the XOR encryption in the first prototype. Additionally, adapt the implementation on the controller side, still in Python.
- Add movement control to the program in order to remotely control it and continue adjusting the control program.

- Implement a ROS 2 control program supporting the symmetric encryption on a widely supported Linux distribution in order to pave the way for simpler and faster integration in the future with ROS 2-compatible controllers.

The main support during the development was the official documentation bundled with the OPEN-R SDK:

- Installation Guide: introduces the main elements of the SDK, how to build and run programs and Remote Processing OPEN-R (this makes it possible to execute OPEN-R program on a remote host which is not AIBO but was not used in this project).
- Programmer's Guide: describes and explains how to develop using OPEN-R, the use of the building blocks, what elements to create or modify in order to run programs and how to read error log outputs. Additionally, available and supported library functions are explained.
- Level2 Reference Guide: describes and explains available classes (as OPEN-R can be viewed as an API to interact with the underlying Aperios OS, predefined classes must be used).
- OPEN-R Internet Protocol Version 4: describes available protocols (TCP, UDP, Domain Name System (DNS) and IPv4) and the OPEN-R building blocks associated.

Other unofficial tutorials were very helpful, such as the *Aibo programming using OPEN-R SDK's* tutorial [46] or the *Introduction to the Aibo programming environment* by Ricardo A. Téllez.

3.5.2 Unauthenticated XOR prototype

TinyConsole

The first prototype (and following versions) of remote console is called TinyConsole as it is inspired by the TinyFTPD program. It is made up of one OPEN-R object also called TinyConsole which opens a custom TCP port and listens for active connections. When a connection is detected, it sends in plaintext a banner which begins an XOR-encrypted session. It can react to three commands:

- PING: will answer "PONG"
- QUIT: will answer "BYE", close the connection, then wait for the next client
- Any message: any other message will be answered with "OK".

While it is fairly basic, this lays the foundations for future network communication and was used as a playground to experiment with different parameters and learn how to develop applications using the SDK.

The prototype's `stub.cfg`, adapted from TinyFTPD's, requires four extra entries:

```
1 ObjectName : TinyConsole
2 NumOf0Subject : 1
3 NumOf0Observer : 1
4 Service : "TinyConsole.DummySubject.DoNotConnect.S", null, null
5 Service : "TinyConsole.DummyObserver.DoNotConnect.O", null, null
```

```

6 Extra : ListenCont() // Invoked when a new TCP client connection is
    accepted -> transitions to connected state // +
    sends the initial banner
7 Extra : SendCont() // Invoked when data has been processed by the IP
    stack + the buffer can be re-used
8 Extra : ReceiveCont() // Invoked when a receive request has been completed
9 Extra : CloseCont() // Invoked when the TCP connection teardown is fully
    completed; resets the state + // posts a
    new listen request for the next client

```

The memory allocation's file used for TinyFTPD was reused for the XOR prototype as is:

```
object tinyFTPD 3072 16384 128 cache tlb user
```

However, if the object was to have very significant memory requirements, for example for encrypting the image stream from the frontal camera of the robot, those parameters might need to be modified.

Apart from those two files and the `Makefile` originating from TinyFTPD, four files are used to set up TinyConsole:

- `ConsoleConfig.h`: defines the TCP port used (7777 was used during development but any uncommon TCP port will work with the exception of 59000 when the `WCONSOLE` configuration is used), the shared ANT buffer size, the XOR key and its length and finally the plaintext banner used to initiate the XOR encryption.
- `TCPConnection.h`: defines the connection state (closed, connecting, etc.), the endpoint for the connection and the ANT buffers (send and receive).
- `TinyConsole.h`: defines the object `TinyConsole`, the four main OPEN-R functions (see Section 2.1.4), private function (detailed in the following sequence diagram), private helper functions to abstract over ANT, two state trackers for the stream cipher (for incoming and outgoing data to ensure the cipher stream remains synchronised even when the client's message lengths vary) and a flag to manage the lifecycle of the connection (initiate its teardown when "QUIT" has been received).
- `TinyConsole.cc`: the core implementation file which dictates the object's runtime behaviour, network state machine and cryptographic processing.

Python console

On the remote computer, a very simple Python script was used to communicate with the robot. It uses sockets to connect to the robot whose IP address and port are hardcoded in the script. During testing, when the robot was connected to the unsecured access point, it always got assigned the same IP address therefore, it was deemed unnecessary to expose the IP and port as command-line arguments.

While the banner is not received, the script waits. Once the banner has been received, it sets its own counters for both the receiving and sending streams, and indicates to the user that commands can be written. Some basic error messages are also available in the event that there was an error when trying to connect or when the robot ended the connection.

A `TinyConsole` interaction between the robot and the remote Python console can be easily visualised using this diagram:

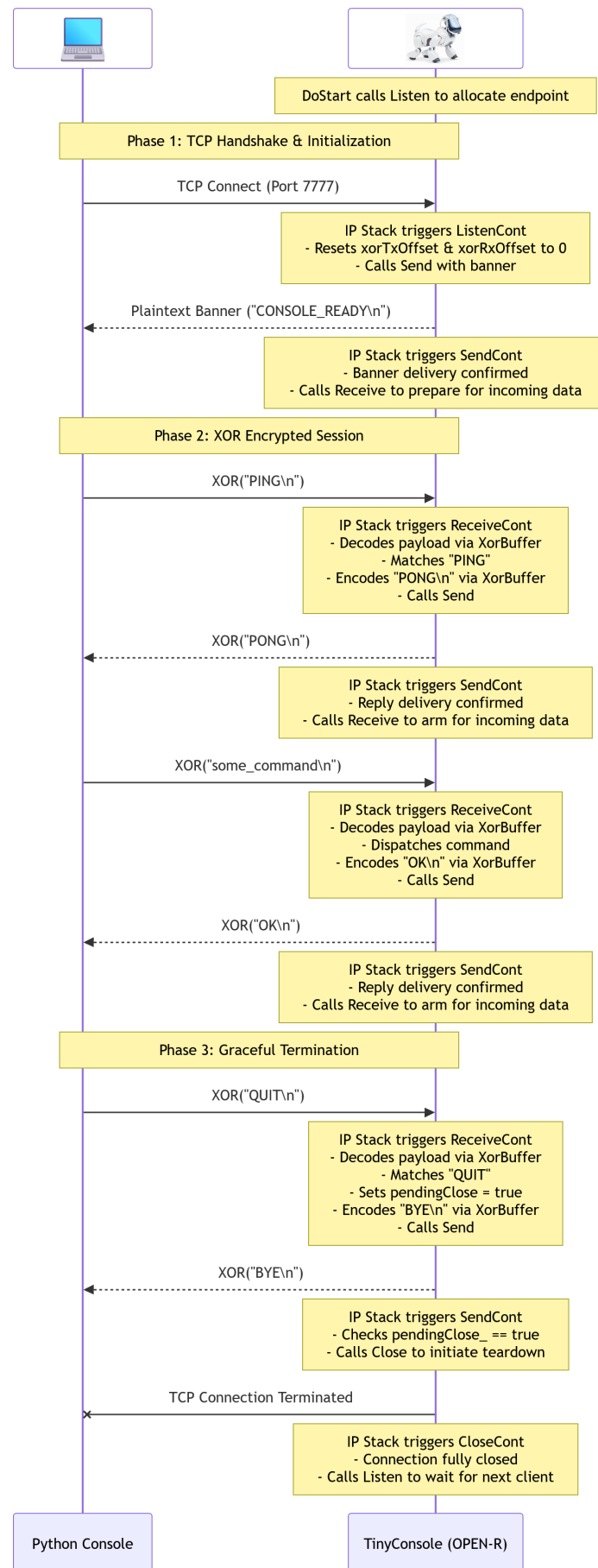


Figure 3.5: Sequence diagram of TinyConsole execution

3.5.3 ChaCha20 and Poly1305

While using asymmetric encryption to exchange symmetric keys in order to subsequently establish a symmetric cryptographic connection would have been the best design to secure the communication between the robot and a remote console, this would have required the ability to generate symmetric keys directly on the robot.

However, one major flaw of Aperios is that it does not have any Random Number Generator (RNG) capability, nor any entropy source. Functions depending on UNIX APIs like `time`, `clock` or `gettimeofday`, which could have been used as a fall-back secondary source of randomness are not available. Another potential source could have been using the low bits of live sensor reads of the robot but this would have meant building and validating from scratch a RNG which is a long and non-trivial task.

For those reasons, the use of symmetric encryption only was the default choice to encrypt the robot's data in transit. A non-negligible drawback of this choice is that if the secret key used to cipher the communication were to be retrieved by an attacker, then all previous and future communication could be deciphered by them (no forward secrecy).

For the choice of the algorithm, as discussed in section 2.2.2, the Ascon family of cipher and, more specifically, the AEAD algorithm is the new National Institute of Standards and Technology (NIST) standard for constrained devices.

The biggest drawback with choosing this cipher was that porting its reference implementation to the legacy MIPS toolchain introduced multiple problems:

- Even if picking the `opt32_lowsize` implementation of Ascon AEAD (32-bit size-optimised in pure C) would have reduced the risk of incompatibility, it still would have been one of the first, if not **the** first time that Ascon AEAD was ported onto a GCC v3.3.2 mipsel cross-compiler. This meant a total lack of existing documentation and debugging resources.
- A non-trivial element of this project is cross-library validation. The naive XOR implementation already relied on this with on one side the legacy OPEN-R object and on the other a modern Python script. While a reference implementation of Ascon in Python developed by one of Ascon's designers exists (`pyascon`), it cannot compare against hardened, verified packages like `cryptography` which give confidence in the client's side implementation.
- Ascon selection by NIST was in February 2023, the initial public draft in November 2024 [47] and the final SP 800-232 in August 2025. While this followed the processes in use at NIST and led to extensive checks, vulnerabilities often only become apparent over time, and in Ascon's case, the timeline is still too recent to ensure complete confidence in an implementation that could still be changed or even be improved.

The decision that was taken was to opt instead for a more conventional stream cipher but to retain this for possible future improvements once it had been demonstrated that secure communication was feasible.

ChaCha20+Poly1305 was chosen for the reasons set out in section 2.2.2.

The biggest danger to keep in mind when implementing and using ChaCha20+Poly1305 is nonce reuse. To encrypt a plaintext, the algorithm takes as input a 256-bit key and a

96-bit nonce. In a situation where a nonce came to be the same as one previously used to create a ciphertext, an attacker monitoring the connection would be able to recover the XOR of the two plaintexts by XORing together two ciphertexts with the same nonce. If the attacker was able to retrieve one of the plaintexts, they would then be able to recover the keystream and as there is no forward secrecy, they could decrypt any previous or future messages.

3.5.4 Choice of the implementation

While there are lots of already existing implementations of ChaCha20 and Poly1305, the choice was made to use an implementation published on GitHub authored by developer Will Glozer. The implementation takes the original ChaCha20 implementation of D.J. Bernstein (designer of both ChaCha20 and Poly1305) and the Poly1305 implementation by programmer Andrew Moon (this particular implementation of Poly1305 is so efficient that it is used in major projects like the Linux Kernel, Google's BoringSSL and libssh2).

The construction of ChaCha20+Poly1305 follows RFC 7539 which has been deprecated in favour of RFC 8439, which fixes some typos, clarifies some language elements and adds a small section of text about the needs for any protocol using the algorithm to include the complete tag to minimise the opportunity for forgery. There are therefore no changes to the actual implementation of ChaCha20+Poly1305, which means that this particular implementation remains valid.

Threats mitigated

The aim of this symmetric encryption and MAC is to secure the communication with a remote host that is communicating with the robot and, more specifically, to mitigate these previously modelled threats:

- DF1 - Spoofing the access point: With this encryption, even if an attacker is able to spoof the access point, they will not be able to intercept credentials, as the data is encrypted, nor to inject commands as this would require retrieving the stream key.
- DF2 - Tampering with the wireless connection: Thanks to Poly1305, any modification to data being exchanged between the robot and a remote computer would be detected as the MAC would no longer correspond.
- DF4 - Exposure of exchanged data: Similarly to DF1, with the symmetric encryption, any captured data could not be decrypted unless the secret key used is leaked.

Post-quantum considerations

Although discussing post-quantum cryptography given the limited power available to the ERS-7 could be considered presumptuous, it must nevertheless be considered, as quantum-resistant cryptography has become an increasingly pressing subject in recent years, with international guidelines recommending 2035 as the latest date from which only Post-Quantum Cryptography (PQC) systems should be used. As time passes, the probability that a cryptographically relevant quantum computer appears grows and systems will become more and more vulnerable to a "store-now, decrypt-later" scenario where attackers store transmission while waiting for a Cryptographically Relevant Quantum Computer (CRQC) to become available [48].

On this subject, both ChaCha20 and Poly1305 [48] are not considered as quantum resistant. *In its configuration of 20 rounds, ChaCha cannot be considered quantum-resistant to a degree comparable to AES-256 (it would require significantly fewer quantum resources to break compared to AES-256)* [48]. To be considered quantum-safe, ChaCha would need approximately 166 rounds. [49] As for Poly1305, as long as the encryption used in conjunction with it can be broken, then Poly1305 is no longer secure in this use case.

This is a known limitation and a solution could be explored once the secure baseline is firmly in place and the remote control features have been expanded.

3.5.5 Asymmetric Authentication with Ed25519

An important limitation of the previously defined scheme is that there is no way to know whether the peer with whom we communicate is really who they claim to be. If the symmetric key was to leak, anybody could pose as the robot or the control computer. In order to mitigate this risk, a signature scheme using public key was determined to be the best choice.

Although there were plenty of options, the decision was made to go with Ed25519 which is an Edwards-curve Digital Signature Algorithm that makes use of SHA-512 and an elliptic curve related to Curve25519. It has also been designed by D.J. Bernstein in collaboration with other researchers and is detailed in RFC 8032. The important thing which makes Ed25519 stand out is that it was designed from the start to be a high-speed signing scheme with high-security (no secret branch conditions and no secret array indices) [50].

As with ChaCha20 and Poly1305, multiple implementations of Ed25519 are freely available but the decision was made to use the implementation defined in TweetNaCl which is an adaptation of the cryptographic library Networking and Cryptography library (NaCl).

NaCl was designed to avoid cryptographic problems such as those that affected OpenSSL. It guarantees [51]:

- No data flow from secrets to load addresses
- No data flow from secrets to branch conditions
- No padding oracles
- No unnecessary randomness

All of this also with the goal of attaining very high speed.

TweetNaCl is an adaptation of the 25 functions of NaCl in order to fit into a maximum of 100 tweets¹ which makes it available from anywhere at any time and is the smallest readable implementation of this high-security cryptographic library [52].

The reason for choosing this implementation of Ed25519 (NaCl's `crypto_sign` function family) mainly boils down to the fact that the implementation is very minimalist and has been implemented by the same team behind NaCl, and therefore D.J. Bernstein, who himself designed Ed25519, making it a reliable source when it comes to the accuracy of the implementation.

¹Following the takeover of Twitter and its rebranding as X, the tweets have since been removed from the account but have been archived by the Wayback Machine.

Threats mitigated

The main goal of the usage of Ed25519 is that once the AEAD session is started, the remote controller will send a message signed with its own private key, and the robot will use the controller's public key to verify the signature. Once the signature is verified, the robot will start the same process but in reverse.

The threat that is therefore mitigated is the following:

- E1 - Spoofing the remote controller: The role of authentication no longer rests with the symmetric key (and anyway, key possession in the scenario of symmetric encryption is a weak authenticator); even if it were to be compromised, only the controller would be recognised as authorised to connect to the robot, and vice versa.

Similarly to ChaCha20 and Poly1305, Ed25519 is not considered to be quantum-resistant as it uses elliptic-curve cryptography which can be broken by quantum algorithms such as Shor's algorithm [48] but the same reasoning as discussed before is applied: only once this baseline is in place and validated could quantum-resistant replacements be investigated.

3.5.6 Secure TinyConsole implementation

This second version of TinyConsole takes the learnings of the previous, naive implementation, and builds upon it using cryptographic elements previously discussed.

Main building blocks

A significant portion of the previously used files were reused nearly as-is or with minimal modifications. These are:

- Makefile: The `-DPOLY1305_32BIT` flag is added so that the compiler uses the 32-bit implementation of Poly1305 (the 64-bit version fails to compile as it requires a native 128-bit integer type to hold the products of its 64×64 multiplies, which the legacy toolchain does not support); and the second important addition is the inclusion of more compiled object files needed for the binary (all files related to used cryptographic implementation).
- `tinyConsole.ocf`: The allocated memory is the same as the declared parameters for the XOR-version of TinyConsole. Given that no errors were encountered, there was no reason to alter them.
- `stub.cfg`: The major difference is that, as the object will need to send joint commands, it must set up a subject called `TinyConsole.Move.OCCommandVectorData.S` but stays with a dummy observer as it does not need to receive back data (network data being handled by ANT).
- `TCPConnection.h`: nothing is changed between the XOR and the secure version as the connection and ANT elements (buffer and endpoints) stay the same.

With the addition of a real subject comes the need to modify the `CONNECT.CFG` which describes the connection between a subject and an object. As joint commands are, as in all Aperios systems, handled by an object, the subject of TinyConsole must send data to the corresponding observer:

¹ [ERS - 7]

```

2 TinyConsole.Move.OCommandVectorData.S
3 OVirtualRobotComm.Effector.OCommandVectorData.O

```

Finally, while `ConsoleConfig.h` is modified to add the necessary keys (ChaCha20's symmetric key, robot public and secret key and client public key) and the handshake context used to build the signature transcript (`Transcript = (HANDSHAKE_CONTEXT || serverNonceSent || clientNonce)`), the file's role remains the same: define important context elements and variables that will be necessary during the object execution.

One point that is important to discuss is why the different sensitive keys (chacha and robot's secret key) are not put into a separate file that could be more easily changed rather than having to modify the code and recompile it. Including the key directly into the program's files also creates a risk where if the files that contain this sensitive information are pushed to a publicly accessible repository like GitHub without replacing the keys value with a placeholder, this information would very quickly be detected and stored by automated detection bots that scan the GitHub API.

The reason for this choice is very simple: OPEN-R SDK and even more the Memory Stick physical hardware gives no options to protect a separate stored file containing secrets. The secrets, if independent from the code of the TinyConsole's object, would have to be stored as a plaintext file. In a situation where an attacker could get physical access to the robot's Memory Stick, they could instantly retrieve the stored secrets.

Although this has already been discussed previously in relation to threats requiring physical access to the robot's Memory Stick in 3.4.3, it is important to emphasise that there is no perfect solution available in this situation. Following some tests using tools such as BinaryNinja and Ghidra, the program's structure and the presence of the keys are not immediately apparent, but this would certainly not be enough to prevent a professional reverse engineer from retrieving the keys. It would simply require a little more effort than with a classic C++ program due to the nature of Aperios.

Finally, regarding key generation, this is done outside as the robot lacks any RNG capabilities. A Python script (`keys_generator.py`) can be used to generate the symmetric ChaCha key or the public and secret key necessary for the Ed25519 signature (in the required formats).

The script uses two Python functions depending on the use case:

- Symmetric key: `os.urandom` which is a Python wrapper that queries the underlying OS cryptographically secure pseudorandom number generator (CSPRNG, `/dev/urandom` on Linux and `BCryptGenRandom` on Windows).
- Ed25519 public and secret key: `cryptography.hazmat.primitives.asymmetric.ed25519` which comes from the cryptography library's hazmat layer which provides developers with ready-to-use implementations to generate, in this case, keys for Ed25519.

Integrated cryptographic primitives

The secure version integrates external C-based cryptographic libraries, the two major additions being:

- The AEAD construction: `rfc7539.c` and `chacha20poly1305.c` provide the implementation of the ChaCha20+Poly1305 cipher.

- The signature construction: `tweetnacl.c` provides the `crypto_sign` (sign a message) and `crypto_sign_open` (verify a message signature with the client's public key) functions necessary for Ed25519 asymmetric identity verification.

Multi-phase states and movements

In order to translate remote commands into physical movements without blocking the robot TCP communication loop, the joint control mechanism is decoupled from the network layer using an asynchronous state machine which relies on four main operational phases:

1. Asynchronous command handoff: when a remote movement command is received, decrypted and validated by the `ReceiveCont` which calls the necessary cryptographic commands, the program does not directly actuate the motors. Instead, the requested actions are stored in the `pendingCmd` state variable and the handler calls the `TriggerReady` function. A status message is generated, encrypted and sent back to the remote controller which frees the network stack which can then wait for the next message. If the robot is idle, a neutral "stay put" frame is sent to the motor observer to start the scheduling loop.
2. Ready event loop: the physical actuation is directed by this function which is triggered every time the robot has consumed the previous block of motor commands. At the beginning of each cycle, it checks for a `pendingCmd`. If a new command exists, it calls `BeginMotion` to interrupt the current action and transition to the newly requested trajectory. This process ensures that new commands are applied at phase boundaries.
3. Linear interpolation: to prevent sudden jerks, joint positions are dynamically interpolated. When a new state begins, `SetupInterpolation` computes the difference between the robot's current joint pose and its target pose array, dividing the delta by a predefined number of steps (for example `GETUP_COUNTER` which must have a different time value than the counter used when moving forward).
4. Continuous Gait chaining: complex movements such as a forward walk require chaining multiple distinct poses together, which is handled by a `switch (motionState)` block inside the `Ready` function. When `AdvanceInterpolation` completes all steps for a phase, it returns `MOVING_FINISH`. The state machine intercepts this flag and sets up the interpolation parameters for the next sequential pose. Completing `MSTATE_WALK_A` triggers the transition to `MSTATE_WALK_B` where this cycle will loop through phases A to H resulting in continuous movement until a `STOP` command interrupts the sequence and transitions the robot back to its broadbase (standing) stance.

Adjusting the robot's joints

Once all necessary OPEN-R related elements were set up, the actual joint movements had to be individually programmed. Most information and precisions for this part came from the "Model Information for ERS-7", Sony's official documentation for this model of the AIBO. One problem that quickly became apparent was that this part of the documentation had some flaws, the first of which being the names given to distinguish between the different leg joints (which were the only ones of significance, given that the aim was to control the robot's movements remotely).

Sony's engineers classified each joint in three categories:

- J1: vertical joint used to rotate the whole leg
- J2: lateral joint used for horizontal or vertical pose
- J3: knee joint used to move the leg

The problem is that only the classifications (J1, J2 and J3) are used and no description nor explanatory diagram help to clarify this.

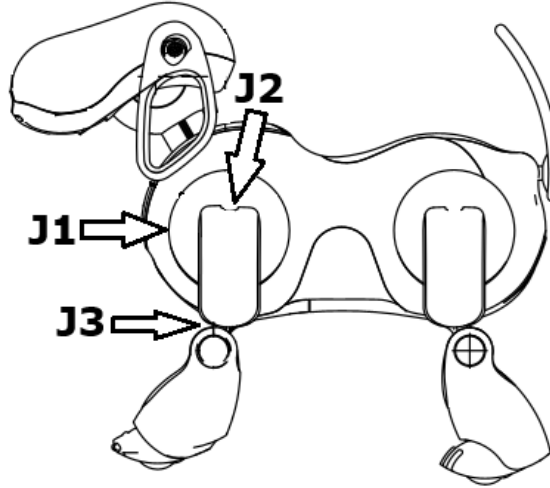


Figure 3.6: Locations of the three types of leg joints
Credit: Model Information for ERS-7

The document lists all primitive locators which must be assigned a value in degrees in order to make the corresponding joint move. The first difficulty was that the initialisation of the joints must be carefully made as a change in position during powering on could cause excessive displacement, which might damage the joint's motor, even if the value specified is within the limits set out in the document.

Furthermore, if the functions relating to the joints are not correctly positioned and initialised in DoInit and DoStart, then nothing will happen when commands are received. For more details, see Appendix 2.

Joint values are assigned using an array in which each individual joint has its opening value in degrees (an integer). For an example of position and assignment, refer to Appendix 3.

In order to provide a sufficiently advanced proof of concept, the robot supports the following movement commands:

- GET_UP: the robot moves from its sleeping position and gets up (standing position)
- REST: the robot goes from standing to lying down
- FORWARD: the robot moves forward
- BACK: the robot moves backward
- STOP: the robot stops moving (not necessary when wanting to move from forward to backward)

Although it is still fairly basic (no camera feed, no left or right turns), the implementation of a Gait engine allowing a robot to move is a research subject on its own [27] and the aim

was not to re-implement all the remote control features, but rather to focus on the security aspect.

Furthermore, as joint gain values must be individually and manually set, creating from scratch movements which correctly coordinate all four legs while balancing the weight of the robot proved to be a bigger challenge than initially thought. Significant time was spent on this subject but in the end, the solution came from a Webots simulation of the ERS-7.

Webots is an open-source robot simulator which supports a wide range of robots, one of them being the ERS-7 which is simulated in a sample environment where it moves across a soccer field. However, the developers of Webots did not reimplement the joint gain corresponding to the walk and instead relied on Motion files which are files used to describe joint movements.

The most useful files retrieved from this robot simulation included those describing the forward and reverse walk movements. Using an application created by DogsBody & Ratchet Software called Skitter (neither affiliated with nor developed by Sony), it was possible to modify, adjust, simulate the AIBO movements and retrieve the joint values so that they can be incorporated into TinyConsole:

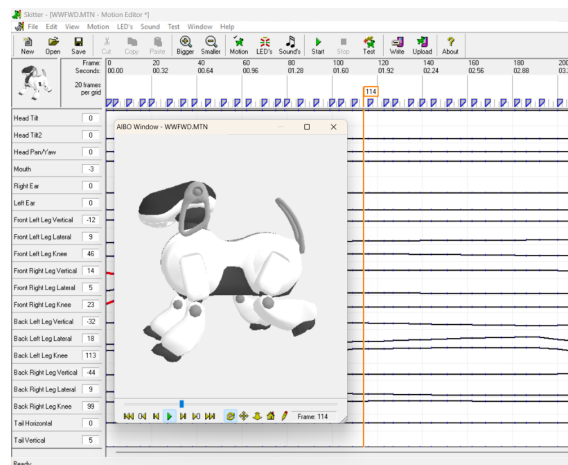


Figure 3.7: Skitter UI

Nonces and non-reusability

Nonce reuse in ChaCha20 and, more generally, in stream ciphers, is a catastrophic risk in the event that a malicious actor was monitoring the network used during the communication. They could simply XOR two ciphertexts with the same nonce to eliminate the keystream and recover the XOR of the two plaintexts.

As the AIBO's lack of hardware entropy prohibits generating random nonces directly from it, a compromise was made: use the remote client to help in generating a nonce that conforms to the ChaCha20+Poly1305 architecture. This construction uses a hybrid 96-bit (12-byte) nonce architecture which complies with RFC 8439.

If the robot was used to generate its own nonces, they would be deterministic and an attacker could predict the sequence or a hardware reboot would reset the sequence to zero.

To avoid this, during the plaintext phase of the TCP handshake:

1. The robot generates a predictable 12-byte baseline nonce consisting of an internal session counter (which resets on reboot but is replaced by a persistent one in 3.5.6) and the client's IPv4 address. This is not secure at all on its own.
2. The remote Python client uses the OS-level Cryptographically Secure Pseudorandom Number Generator (CSPRNG) via a call to `os.urandom` to generate 12 bytes of high-entropy randomness, which is sent to the robot in cleartext. This is not a problem as if an attacker is able to accurately know what the nonce is for each message, this would give them no advantage as long as they do not know the symmetric key.
3. Both endpoints bitwise XOR the first 8 bytes of their respective nonces together.

The resulting 8-byte array forms the `session_prefix` which guarantees that every TCP session uses a globally unique cryptographic baseline despite the AIBO's limitations.

Once the baseline is established, every subsequent AEAD frame must still use a unique 12-byte nonce. This is done by constructing the nonce in memory immediately before calling the `rfc7539_init` cryptographic context. The 12 bytes are divided into two blocks:

1. Bytes 0-7 (Session prefix): 8-byte XOR combined entropy established during the handshake which remains static for the duration of the TCP connection.
2. Bytes 8-11 (Message counter): A 32-bit message counter that starts at zero upon connection and increments with every AEAD frame encrypted or decrypted.

As the TCP console operates in half-duplex (while the nonce space was designed to be safe even for full-duplex operation) with both the robot and the client encrypting and sending messages independently, they cannot use the same 32-bit counter as it would generate the same nonce for their first messages sent. To prevent this, the nonce construction is implemented using a directional bit-toggle: inside the `AdvanceNonce` and `build_nonce` functions, the 32-bit message counter reserves its most significant bit (Most Significant Bit (MSB) which is bit 31) as a direction flag:

- Client-to-robot (RX or Receive): the client transmits with the high bit cleared.
- Robot-to-client (TX or Transmit): the robot transmits with the high bit set via a bitwise OR operation (`mc |= (isTx ? (1U << 31) : 0)`).

With this construction, it is guaranteed that a nonce used by the robot to encrypt a response cannot match a nonce used by the client to encrypt a command.

However, the limitation of the absence of any RNG capabilities does leave one gap: the proposed scheme guarantees that two honest sessions never share a keystream but it does not stop an active attacker from replaying a session recorded earlier.

Replay resistance across reboots

The element that keeps two sessions apart comes entirely from the client's `os.urandom` contribution: the baseline the robot supplies (`serverNonceSent`) is derived from a session counter that resets to zero on reboot and the client's IPv4 address. This would be sufficient against a Man-in-the-middle (MITM) and against impersonation of the robot but not against replay of the client-to-robot direction.

When recording a complete session, the cleartext `serverNonceSent`, `clientNonce`, client's

Ed25519 signature and the encrypted command frames can be stored. After the robot reboots, which is bound to happen as its battery can last around 1.5 hours, the session counter returns to its initial value (zero).

The crux of the matter is that if the attacker reconnects from the same IPv4 address (spoofing), the robot regenerates exactly the same `serverNonceSent`. The attacker only has to replay the recorded `clientNonce`, rather than generating a fresh one, and the robot will reconstruct an identical transcript for which the recorded client signature checks out (without the attacker ever possessing the client's private key). Once the handshake finishes, the recorded AEAD frames replay with matching nonces and valid Poly1305 tags, making the robot re-execute the captured command sequence.

The client's randomness cannot close this gap as replay resistance requires that the affected party (in this situation, the robot) contributes freshness that an attacker cannot force to repeat or retain state of what has already been accepted.

This would require one of three resources:

- A source of entropy: This is a dead end as the robot does not have any options capable of generating sufficient entropy.
- A trusted clock: This is also a dead end as stated in the Programmer's Guide, functions like `time` rely on UNIX APIs which are not available as Aperios is not a UNIX-based OS.
- Persistent non-volatile state: This is the best compromise although it also introduces another limitation. When needing to write data on the Memory Stick, it means that the switch that blocks writing must be off. This switch ensures that, unless an attacker had physical access to the robot, they could not modify the TinyConsole program.

The persistent non-volatile state takes the form of a monotonic session counter persisted on the Memory Stick. Unlike the volatile session counter, this value is never reset: it is read at start-up, incremented once per connection, and written back before the handshake completes (a crash or power loss can only skip values forward and never reuse one). The counter is bound into both the nonce baseline, specifically the session-counter word of `serverNonceSent`, and therefore into the signed transcript which already covers `serverNonceSent`.

Because the counter appears inside the material signed by both parties, a replayed client signature (produced over an older counter value) no longer matches the transcript the robot reconstructs for the new session and `crypto_sign_open` will fail. The attacker cannot repair this since forging a signature over the new counter value would require the client's private key.

The counter is not secret and can be stored in plaintext as its security rests on being authenticated and monotonic rather than confidential. The write is made atomic through a two-slot scheme with a validity field, reusing the same Memory Stick write path that was already used by the Exception Monitor log.

The client retains the last counter value it accepted and rejects any session in which the value does not strictly increase (this catches rollback attempts over the network). If for any reason the counter cannot advance (bug, physical switch on, etc.), the robot fails closed.

Finally, an attacker with physical access to the robot could roll the counter back directly but such access already exposes the stored keys (DS1 & DS3) and lies outside the scope of this work.

Signature and authentication

The authentication that is provided by Ed25519 protects the TCP connection: the OPEN-R application will refuse to process any symmetric AEAD frames until both the robot and the client have authenticated themselves via signatures.

The architecture of Ed25519 requires the public keys to be exchanged in advance; if the keys were exchanged via the TCP connection, there would be no way of knowing whether the robot or the client really is who they claim to be. This would require a Public Key Infrastructure and is out of scope for this implementation.

The C implementation of the NaCl cryptographic library, `tweetnacl.c`, is used to sign the messages. During the handshake, neither endpoint signs the random nonces directly. Instead, they both sign an identical, reconstructed memory buffer (transcript). It is defined as `Transcript = HANDSHAKE_CONTEXT || serverNonceSent || clientNonce`.

This serves as a domain-separation tag: the signature is bound to this specific protocol and prevents cross-protocol replay attacks (an adversary cannot capture a signature from this TCP console and reuse it to authenticate against a potential future service on the robot as it would only validate against this exact context string).

In the first phase of the authentication, the robot must prove its identity to the remote client. Inside the `SignHandshake` function, the robot constructs the transcript and passes it to `crypto_sign` alongside the securely compiled `ROBOT_ED25519_SK` (robot's private key). TweetNaCl uses a combined API, outputting a buffer containing the 64-byte signature followed by a copy of the original message. TinyConsole extracts only the first 64 bytes (detached signature) to minimise the overhead and transmits it to the client which then verifies it against the robot's already stored public key (`ROBOT_PUBKEY_HEX`), before trusting the connection.

Once the client has verified the robot, it generates its own 64-byte Ed25519 signature over the exact same transcript using its private key and transmits it back to the robot. The network state machine transitions to the `RX_CLIENT_SIG` phase to intercept this payload and upon receiving the signature, the `VerifyClientHandshake` function executes those steps:

1. It reconstructs the exact transcript using the nonces exchanged earlier in the handshake.
2. In order to satisfy TweetNaCl's combined-format requirement, it dynamically allocates a buffer (`sm`) and concatenates the received 64-byte client signature with the locally reconstructed transcript.
3. Finally, it calls `crypto_sign_open`, passing the combined buffer and the pre-authorised `CLIENT_ED25519` public key defined in `ConsoleConfig.h`.

If `crypto_sign_open` returns 0, the client's identity is verified and the state machine safely moves on to `RX_FRAME_HEADER` to begin accepting ChaCha20+Poly1305 encrypted

command traffic. However, if the verification fails, the connection is terminated and an error message detailing the origin of the failure is printed.

Frame sizes

Despite all those security considerations and even if all sensitive data are fully encrypted, an important thing is to give as little information as possible to a potential attacker. However, if data are encrypted but the size of frames changes depending on processed commands, an attacker could still analyse the traffic and guess states of execution.

This is why once both client and robot have sent back their signature and the AEAD session has started, all frames have a fixed size of 274 bytes (2-byte block size || 256-byte ciphertext || 16-byte tag) where `ciphertext = real_length || plaintext || zero padding`. A variable `PAD_BLOCK` is used to define the size of a message. Before encryption, each plaintext command or response is padded up to `PAD_BLOCK`.

The added benefit is that frame sizes do not carry any remaining information and no fingerprinting of any type can be done on this parameter. Making the size of a frame a variable rather than a constant is a choice to enable easier forward compatibility in case larger commands are introduced later. The frame size is fixed at build time but is carried in a header field so it can be enlarged later without significant changes.

The trade-off is found in bandwidth: every frame leads to the padding of the chosen padding size which is why the smallest value that safely covers the command set must be chosen instead of an arbitrarily large value.

This also does not protect against frame count or timing which would require sending frames on a fixed schedule using dummy frames when there are no commands to send. The drawback is that it adds increased latency which may be a problem when controlling a robot's movements in real time.

Adaptation and improvements of the console script

The cryptographic elements of the remote controller having been already thoroughly detailed in the previous sections, this section serves as a summary of the structural transformation of the Python console script.

To support the secure TinyConsole implementation, the script had to evolve from a basic plaintext socket tool into a cryptographic peer mirroring modifications made from the XOR-version of TinyConsole into its secure, final version. Three main improvements were made:

1. Integration of Python cryptography: to mirror the custom C implementations of the OPEN-R object (`rfc7539.c` and `tweetnacl.c`), the script integrates the `cryptography` package with its Ed25519 implementation for handshake signature generation and verification, and the `ChaCha20Poly1305` AEAD primitive for the continuous stream. This ensures cross-platform secure compatibility between a modern OS (OS agnostic implementation) and the Aperios environment.
2. Byte-aligned state machine: the original XOR prototype relied on blocking buffer reads that looked for `\n`. In the secure version, this part of the script was entirely rewritten to execute a byte-aligned reading sequence that mirrors the robot's

ReceivePhase enumerations. The client executes exact-byte reads from the TCP socket: 26 bytes for the plaintext banner and server nonce, 64 bytes for the Ed25519 signature and 2 bytes for the AEAD frame header (followed by the length specified in that header plus the 16-byte Poly1305 tag).

3. Directional nonce tracking: the script implements the client-side counterpart to the bidirectional nonce by maintaining its own internal 32-bit counter, ensuring the MSB remains strictly cleared (0) when encrypting outgoing commands and expecting the highest bit to be set (1) when decrypting incoming responses from the robot. This ensures that both endpoints can communicate in full-duplex without risking nonce collision.

Secure TinyConsole flow of execution

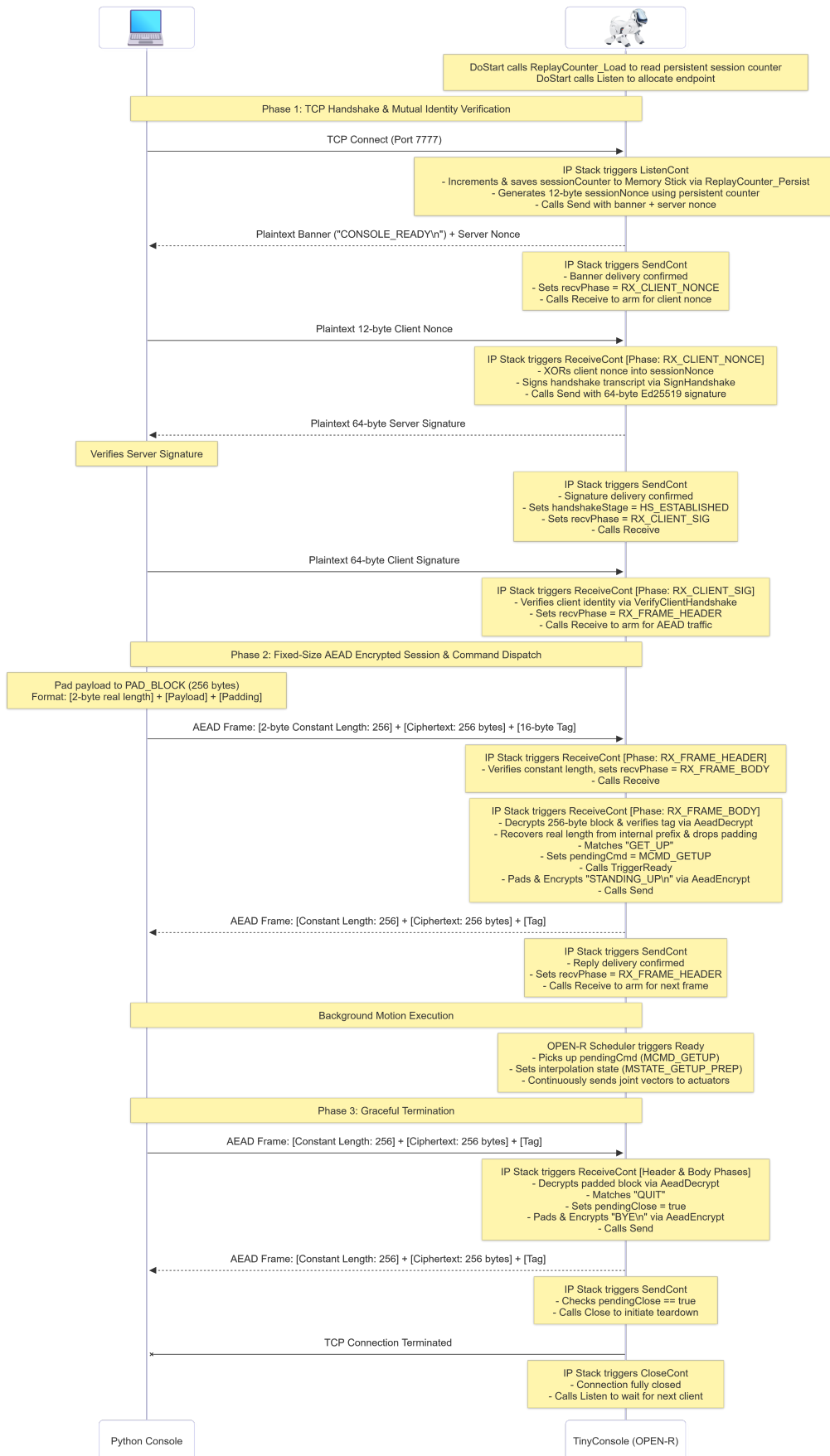


Figure 3.8: Sequence diagram of secure TinyConsole

3.6 ROS 2 integration

While the previous work on the AIBO's OPEN-R program is of interest from a security perspective, it stays limited in terms of its usefulness. But as the TinyConsole 'protocol' is agnostic to what drives it, it can be reached and reused even by modern robot operating systems such as ROS 2.

This section goes over the reason behind the choice of ROS 2, where it sits with respect to the security layer, the components that make up the implementation and finally presents a keyboard remote-control interface built on top of it to showcase the scope for adapting the methods used to control the robot.

3.6.1 Usefulness and choice

It is worth noting that nothing in this integration of ROS 2 is functionally necessary. The previously described console already sends individual commands to the robot and the cryptographic layer lives entirely in that client and the OPEN-R program. Another Python script could have been implemented in order to use a keyboard to send encrypted commands to the robot without ever needing the ROS 2 layer which can be considered overhead (package, build system with `colcon`, launch files, etc.) taking into account the components and constraints compared with what a single script could provide.

The reasoning is not the current feature added but the integration it makes possible. Even a 20-year-old robot, akin to abandonware, can be addressed through a modern, widely adopted robotics middleware (one important element is that, despite its name, ROS is not intended to replace the operating system of the robots with which it is used).

Once the ERS-7 'speaks' the standard `Twist` velocity interface on `cmd_vel`, any component of the ROS 2 ecosystem can command it without knowing anything about TinyConsole, OPEN-R or Aperios: a joystick through the `joy` node, an autonomous navigation stack, `ros2 bag` for recording and replaying command sequences or `RViz` for visualising the robot's published state. Changing the input device requires no change to the bridge and this is the concrete contribution.

However, no existing package is sufficiently suitable to meet the requirements without adaptations. The `teleop_tools` suite (`key_teleop` the keyboard controller and `joy_teleop` the node to map joystick controls to robot commands) is the ROS 2 solution for mapping inputs to commands but its nodes publish `Twist` velocities which are not suitable for the AIBO's commands.

To resolve this 'translation' issue, a node was written, reusing the standard velocity interface for locomotion while handling postures separately.

3.6.2 Security considerations

An important property of the design is that ROS 2 sits above the security and does not touch the crypto layer but its default discovery mode would widen the attack surface which is why it is bound to localhost (an attacker on the same unencrypted network would be able to send commands to the topics exposed by ROS). The ChaCha20+Poly1305 and the Ed25519 authentication are the same elements from the previous console and the node only

instantiates that client. This means that the middleware layer is transport-independent with regard to confidentiality and integrity.

Three choices prevent the integration from reducing this boundary:

1. The key material is never embedded in source as it is supplied at launch through a YAML parameter file (kept out of the online repository) while the non-sensitive parameters (robot address, port and teleoperation elements such as the time remaining before a timeout) remain basic launch arguments.
2. The node refuses to start if any key parameter is missing or incorrect rather than using an unauthenticated connection.
3. `ROS_AUTOMATIC_DISCOVERY_RANGE` is set to `LOCALHOST` to prevent ROS 2 entities (topics, services and actions) from being visible to other computers on the local network.

3.6.3 Building blocks

First and foremost, ROS 2 has many versions. For this implementation, the choice fell on the Jazzy Jalisco distribution which is supported until May 2029. The operating system used to develop, test and compile was an Ubuntu 24.04 LTS virtual machine to avoid potential compatibility issues with Windows 11.

The package `aibo_bridge` is an `ament_python` package whose central component is the node `aibo_bridge_node`. The node wraps the authenticated console client and exposes the robot through four topics:

1. `/aibo_bridge/command`: direct command pass-through (`GET_UP`, `REST`, `STOP`, etc.)
2. `/cmd_vel`: teleoperation-compatible velocity stream (only `linear.x` is used)
3. `/aibo_bridge/response`: plaintext reply returned by the robot
4. `/aibo_bridge/connected`: link status, republished periodically

As `TinyConsole` is half-duplex (client always sends first and every send must be followed by exactly one receive before the next send), the node serialises all authenticated exchanges through a single lock in order to limit to one encrypted transaction being sent at a time. This causes a latency when commands are issued far apart in time: each command incurs a full send/receive round trip on the robot (around one or two seconds).

Velocity commands received on `/cmd_vel` are translated into the AIBO's '*vocabulary*'. A positive `linear.x` above a certain dead-band value (configurable in `aibo_bridge_launch.py`) maps to FORWARD while a negative value below the dead-band maps to BACK. Anything within the dead-band maps to STOP.

The translation is debounced: a command is only forwarded when the motion state changes so that a continuous streamed velocity does not flood the robot with repeated frames. As there are no primitives to make the robot turn, only forward and backward locomotion are mapped.

Connection management is handled by a periodic timer that re-checks the link (the value of the timer can also be adjusted). If the link is down, the node attempts to reconnect. The

current status is continuously published on `/aibo_bridge/connected` at regular intervals. This was verified and tested by manually disconnecting the robot from the access point: once the timer interval elapses, `/aibo_bridge/connected` reports `false` and the node resumes the connection automatically when the robot is reconnected.

It also causes this behaviour: when issuing the `QUIT` command, the session is closed but it is reconnected within one interval. To fully stop the session, the shortcut `CTRL + C` must be used.

Two files are intentionally left empty:

- `__init__.py`: Used to mark the source directory, without it `setup.py` could not discover the modules or their entry points.
- `resource/aibo_bridge`: Ament resource-index entry which is installed by `setup.py` to `share/ament_index/resource_index/packages/aibo_bridge`. This is how ROS 2 registers that the package exists in the index.

To build and launch the workflow, the build must be invoked from the workspace root rather than from inside the package directory (otherwise a nested build tree is created and the resulting executables are installed under a prefix that ROS 2 does not search):

```
1 sudo apt install python3-cryptography # runtime dependency of the console
   client
2
3 # Run from the workspace root
4 colcon build
5 source install/setup.bash
6
7 # Launch and override the robot address if needed. Key material is read
8 # from the (uncommitted) YAML parameter file referenced by the launch file
9 ros2 launch aibo_bridge aibo_bridge_launch.py robot_ip:=192.168.1.124
10
11 # Issue a single command
12 ros2 topic pub --once /aibo_bridge/command std_msgs/msg/String "data:
   GET_UP"
13
14 # Observe the robot's replies and the link status
15 ros2 topic echo /aibo_bridge/response
16 ros2 topic echo /aibo_bridge/connected
```

A clean rebuild, when required, is done either manually by removing the `build`, `install` and `log` directories or by using the `colcon clean workspace` command provided by the `colcon-clean` extension (not included in the base installation of ROS 2).

3.6.4 Keyboard remote control

The most direct and evident demonstration of the bridge is manual teleoperation. The stock `teleop_twist_keyboard` node can drive the robot through `/cmd_vel` but, as has already been said, it publishes only `Twist` velocities and therefore cannot express posture commands such as `GET_UP` or `REST`.

A dedicated node, `aibo_teleop_key`, was implemented in order to fill this gap with an intuitive key map: up and down arrows send the one-shot postures `GET_UP` and `REST`;

left and right arrows drive FORWARD and BACK through `/cmd_vel`; `Enter` issues STOP and `q` exits.

What distinguishes this from a naive command sender is **stop-on-release**: the robot will move only while a key is held and will stop as soon as it is let go. This reuses the existing velocity translation and debouncing.

The difficulty with this twist is that a terminal will only report key presses (not releases). Therefore, the node relies on the OS's keyboard auto-repeat. While an arrow is held, the key repeats and each repeat re-publishes the motion velocity. When the key is released, the repeats cease. If no key arrives within a `key_timeout`, the node publishes a zero velocity which sends a STOP command to the robot.

The inherent trade-off is the system's initial auto-repeat delay (a few hundred milliseconds) between the first key press and the moment the repeats begin. If `key_timeout` is set below this delay, the node will publish STOP in the short time between repeats and a short stop-and-restart. A timeout of around 0.6s eliminates the stutter with the trade-off of a correspondingly small delay before the robot stops on release.

An additional safeguard aligned with the stop-on-release goal is that the robot stopping does not depend only on the teleoperation node staying alive. A STOP will be sent if the node is commanding motion and no velocity message has arrived within a timeout. This helps to avoid a situation where the robot would continuously walk while the teleoperation process has unexpectedly terminated.

Here is how to launch the node:

```
1 # Default settings
2 ros2 run aibo_bridge aibo_teleop_key
3
4 # With explicit speed and release timeout
5 ros2 run aibo_bridge aibo_teleop_key --ros-args -p speed:=0.5 -p
   key_timeout:=0.4
```

Taking into account these various factors relating to ROS 2 (bridge and keyboard control), this strengthens the claim that a robot whose native control interface predates ROS entirely can be operated through modern, standard robotics tooling whilst using fully encrypted and authenticated communication.

3.7 Final check of the STRIDE threats status

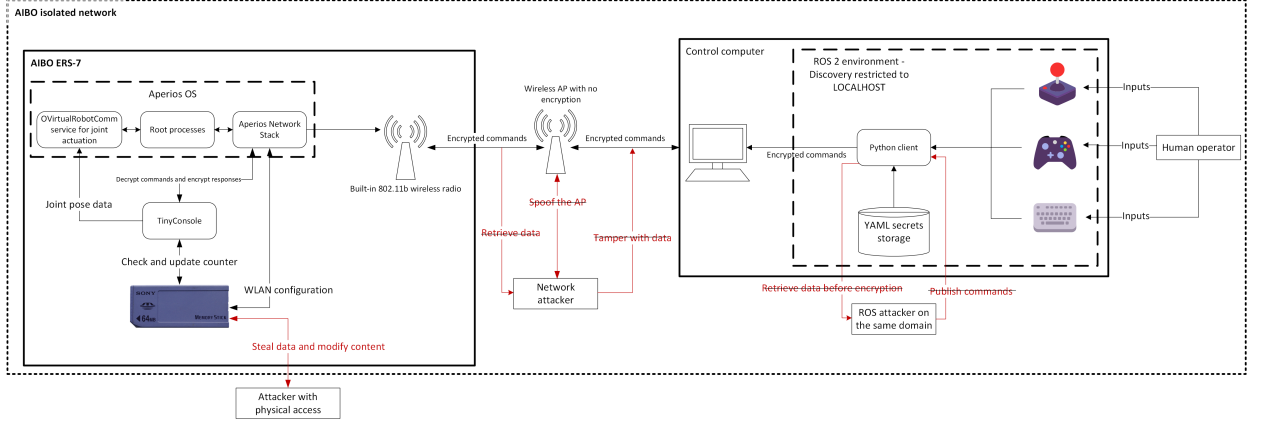


Figure 3.9: Final AIBO data flow diagram

With this new secure implementation and ROS 2 integration, all high-priority threats have been mitigated:

- Exchanged data are encrypted once the AEAD has started, previously exchanged data are in plaintext but do not give an attacker any advantage.
- If an attacker tries to spoof either of the actors (robot or remote controller), even in a situation where the symmetric key was to leak, both the OPEN-R program and the client console would detect that the signature is missing or is incorrect and would stop the connection (E1, DF1).
- Tampering with the wireless connection by modifying exchanged packets would also be detected as the Poly1305 tag which comes with each packet would not match anymore.
- Previous communications cannot be replayed as the static counter is continuously increased and an anti-rollback mechanism is implemented on the client side.
- The ROS 2 integration does not extend the network attack surface beyond the control computer, since Data Distribution Service (DDS) discovery is restricted to localhost. This means that an attacker on the same domain cannot read the commands before they are encrypted and, crucially, cannot publish commands on ROS 2 topics. There is a residual exposure but it originates from the processes already running on that host.

Medium-priority threats are not mitigated and the same applies to low-priority. This is a consciously accepted residual risk as the goal was laid out in advance: high-priority threats were the most important to mitigate as they were the most significant exposure allowing external actors to get information about the execution of the robot.

One final element to note: while in the event of a symmetric key leak an active attacker would be detected because they lack the signature, a passive attacker monitoring the network connection would be able to decipher exchanged messages. The critical issue remains physical access to the Memory Stick and the encryption keys stored in the control computer's console which could cause a leak of stored keys and jeopardise the whole security.

Chapter 4

Testing and measurements

While this secure version of TinyConsole is considered to be functional and no bugs were found in its latest version, this is not enough by itself. Although it is very difficult to be absolutely certain that the implementation is free of bugs and of undetected vulnerabilities (at least those that were not considered in the STRIDE analysis), it is possible to raise our trust in the implementation by doing a series of tests.

Furthermore, and as was the intention from the very beginning, it is vital to measure the impact of this implementation. This being said, there is a need for clarity: it is not feasible to measure the CPU load or RAM usage as OPEN-R does not have any capacity to get this type of data (if we had direct access to the CPU or the Aperios OS, that would be a different discussion).

What is possible, however, is to measure the physical storage impact (size of the compiled binary) and the network latency between the insecure XOR-encrypted version of TinyConsole and its final form (ChaCha20+Poly1305, Ed25519 signature and replay counter).

4.1 Testing

The most important element to test is, of course, the correctness of the implementation of the AEAD scheme. As ChaCha20, Poly1305 and Ed25519 are tried and tested algorithms and their building blocks, as explained in the previous chapter, are either implementations created by the algorithm's designer or widely used implementations, this is not where mistakes should be searched.

On the other hand, the glue that is holding those constructions together may contain errors. Whether it is the implementation combining ChaCha20 and Poly1305 or its integration into TinyConsole, that is where the problem lies. To build trust, multiple Python scripts were created:

- **aibo_mitm.py**: MITM test suite for the ChaCha20+Poly1305/Ed25519 protocol with five functions:
 1. **passthrough**: Produce capture logs when used on an AP through which the robot and client connect (for example an attacker's computer used to spoof the genuine AP). The logs can be used with other scripts. If the two endpoints cannot communicate, then there is a problem in the implementation. The drawback for the current implementation is that the console client needs to specify the IP of the AP instead of the true IP of the robot for this script to log traffic. As the AP only acts as a bridge, it does not have direct access to the frames going through it.
 2. **replay**: Duplicate the client's first command frame to the robot to verify the correct implementation of the robot's counter. The expected result is that the

robot does not execute the duplicated command and drops the connection.

3. **tamper**: Flip one bit in the first client to robot frame (it is possible to select either the ciphertext or the 16-byte tag). The AEAD verification must fail, the robot must refuse the frame, and the connection must be dropped. It is mainly used to prove the integrity property of the connection: it is not possible for an attacker to modify the transmitted information as the Poly1305 MAC will not match.
 4. **mitm-key**: Tamper with the robot's signature during the initial handshake (equivalent to substituting a generated key). The client checks against its copy of the robot public key and drops the connection as it will not correspond anymore.
 5. **rst**: After N frames (value can be chosen), the script sends a TCP RST packet to both endpoints to force a connection drop (not FIN as it should not be a graceful termination).
- **replay_session.py**: Use a client-to-robot communication captured from an earlier session using the **passthrough** mode of **aibo_mitm.py** and replays it into a fresh TCP connection to the robot. It should fail as the robot issues a new handshake nonce for every connection (strictly increasing monotonic nonce stored on the Memory Stick) which means that the Ed25519 signature verification will fail.
 - **rst_inject.py**: Use the same principle as **aibo_mitm.py**'s mode **rst** but in a situation where the attacker does not even need to spoof the connection, all they need to do is be on the same network. It will also send TCP RST packets to both endpoints.

The setup to carry out the attack was to use a laptop running Arch Linux with no defined **nft ruleset** nor firewall and to run from it an unsecured hotspot. All previously defined scripts were run on the laptop.

The execution of **aibo_mitm.py** can be seen from the point of view of the client:

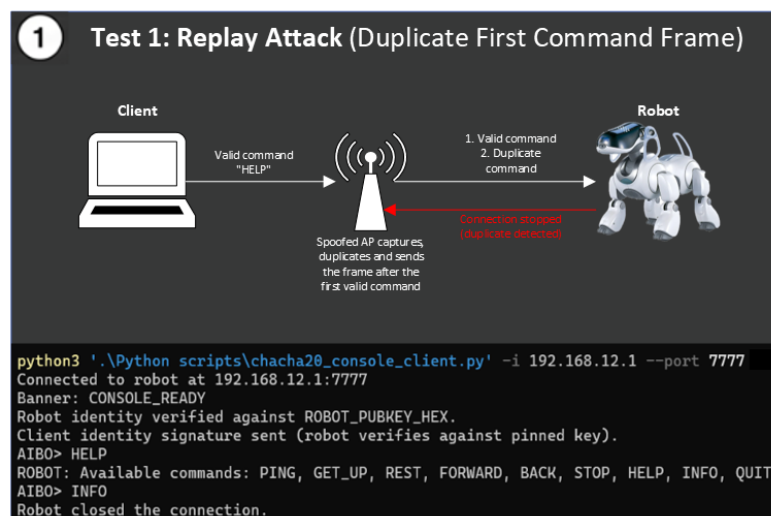


Figure 4.1: Replay mode

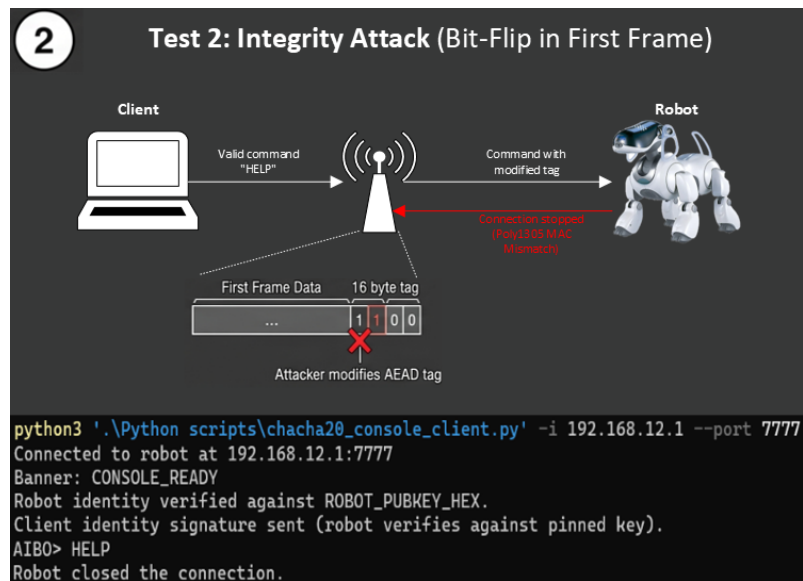


Figure 4.2: Tamper mode

In both **replay** and **tamper** modes, no proper error message can be seen from the client's side. The reason is that the only way to see the error messages coming from the robot would be via the telnet debug console but in a production environment, it must be deactivated as it is a security risk.

However, it is clear – and the tests have been repeated several times – that the security measures in place work well in these situations. The robot terminates the connection in both cases.

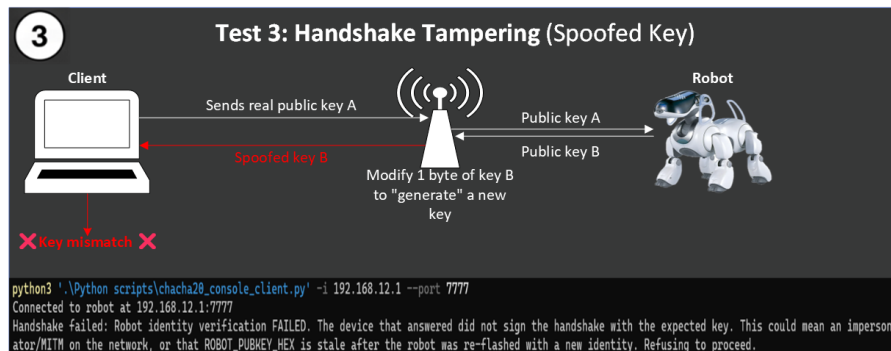


Figure 4.3: Interception of keys mode

In this third attack, the client will see an error message detailing that the received public key does not correspond to its stored version. The script will then drop the connection. The test can also be done the other way by modifying the public key of the client when it goes through the access point and the result would be the same, except that the client would only see "Robot closed the connection".

The `replay_session.py` script was used against the robot to verify the correctness of the implementation of the strictly increasing nonce stored on the Memory Stick. Without it, the robot would have accepted the communication and replayed the same received commands as there would have been no way to verify that this was an older session.

The result, carried out over multiple tries and multiple recorded sessions, was that the robot always closed the connection and never carried out any previous commands. This was the expected outcome.

Finally, for the `rst` mode of `aibo_mitm.py`, it successfully stopped the communication between the robot and the client by resetting the TCP connection, here after allowing 5 frames to run (this includes the 2 frames needed to establish the connection).

```
python3 './Python scripts\chacha20_console_client.py' -i 192.168.12.1 --port 7777
Connected to robot at 192.168.12.1:7777
Banner: CONSOLE_READY
Robot identity verified against ROBOT_PUBKEY_HEX.
Client identity signature sent (robot verifies against pinned key).
AIBO> 1
ROBOT: OK
AIBO> 2
ROBOT: OK
AIBO> 3
Robot closed the connection.
```

Figure 4.4: Connection reset

The observed result was the same when using `replay_session`, the only difference being that it was not necessary for the client to indicate the IP address of the access point.

Although this is related to the DF5 threat that is described in Section 3.4.2 and it is unresolved, the current implementation of TinyConsole never had the goal to protect against DoS-type attacks. This would require a different approach and would involve inspecting incoming packets. This could even be the purpose of another OPEN-R object responsible for inspecting the robot's incoming packets, which would act as a firewall.

While those scripts do not ensure that the implementation is perfect, it does increase trust by demonstrating, through previously identified threat scenarios, that the implementation is resilient to them and therefore that no serious security issues were found. It would be worthwhile to continue using those for future improvements in order to prevent any reversal.

4.2 Measurements

Another way to build trust in the implementation is by relying on measurements:

- `nonce_audit.py`: Reads every session's `.json` file produced by `aibo_mitm.py` and checks the nonce construction across sessions for:
 - `session_prefix` reuse (bytes `[0..7]` which is `robot_nonce[:8] XOR client_nonce[:8]`).
 - Full `robot_nonce` or `client_nonce` reuse.
 - Monotonicity of the robot's persistent counter (nonce bytes `[0..3]`).
- `cipher_sample.py`: Collects every 256-byte ciphertext block from the captured session logs and writes them concatenated to a single binary file. Then, using `ent`, prints (all descriptions are taken from John Walker / Fourmilab):
 - Entropy: *"information density of the contents of a file (expressed as a number of bits per character). "*
 - Chi-square: *"Express how frequently a truly random sequence would exceed the calculated value. If the percentage is greater than 99% or less than 1%, the*

sequence is almost certainly not random. Between 99% and 95% or between 1% and 5%, the sequence is suspect. Between 90% and 95%, it is almost suspect."

- Arithmetic-mean: *"Result of summing all the bytes and dividing them by the file length. If the data is random, it should be close to 127.5."*
- Monte-Carlo Value for Pi: *"Each successive sequence of six bytes is used as 24 bit X and Y co-ordinates within a square. If the distance of the randomly-generated point is less than the radius of a circle inscribed within the square, the six-byte sequence is considered a "hit". The percentage of hits can be used to calculate the value of Pi. For very large streams, the value will approach the correct value of Pi if the sequence is close to random."*
- Serial-correlation: *"Measures the extent to which each byte in the file depends upon the previous byte. In the context of random sequences, this value will be close to zero (can be positive or negative). Non-random byte stream will have a result on the order of 0.5. For wildly predictable data, it will exhibit serial correlation coefficients approaching 1."*
- Modifications to both `xor_console_client.py` and `chacha20_console_client.py`: Addition of a benchmark *mode* to measure the RTT (several parameters are available, such as the number of commands to be measured, the specific command to be sent, the number of frames to be ignored, and the option to save the measurements to a CSV file).
- `plot_latency.py`: A simple script used to plot values from the CSV files generated by the client's script when used in benchmark *mode*. The script is also used to generate a Latex table to display key values related to the selected CSV file(s).

4.2.1 Verification of the nonce construction

After using the scripts described in Section 4.1 and running a series of sessions to ensure that sufficient metrics were recorded, `nonce_audit.py` was applied on those captured sessions.

```

1 sudo python3 nonce_audit.py --dir captures/
2 Audited 17 session(s) from captures/
3
4 [ok]    session_prefix (64-bit): no reuse across 17 session(s)
5 [ok]    robot_nonce (96-bit): no reuse across 17 session(s)
6 [ok]    client_nonce (96-bit): no reuse across 17 session(s)
7
8 [ok]    robot persistent counter strictly increasing (11 .. 34)
9
10 --- analytical bound ---
11 prefix space      : 2^64
12 sessions observed (k) : 17
13 P(any prefix collision) = k^2 / 2^65 = 7.833e-18
14 Reuse becomes ~50 percent likely only near k = 2^32 (4.3 billion) sessions.
15
16 RESULT: no nonce reuse observed; construction is sound.
17 Cannot be considered a proof that the scheme is secure

```

All tests are successful, however in view of the prefix space and the birthday problem, it would require far more than 17 sessions to ensure that the nonce is constructed correctly

and that there is no risk of a collision before the specified limit.

First and foremost, this helps to ensure that the implementation is correct in practical terms.

4.2.2 Ciphertext analysis via ent

The second key test concerns the construction of ciphertexts and their supposed randomness:

```
1 sudo python3 cipher_sample.py --dir captures/
2 [*] wrote 25088 bytes (98 ciphertext blocks) to cipher_sample.bin
3 [i] Note: this is a small sample. `ent` is fine here, but dieharder and
   NIST STS need many megabytes to be meaningful.
4
5 [*] running: ent cipher_sample.bin
6
7 Entropy = 7.992854 bits per byte.
8 // Maximum theoretical value = 8 bits/byte
9
10 Optimum compression would reduce the size
11 of this 25088 byte file by 0 percent.
12 // Indicates that there are no discernible patterns or redundant structures
   for an algorithm to compress
13
14 Chi square distribution for 25088 samples is 247.82, and randomly
15 would exceed this value 61.47 percent of the times.
16 // Percentage outside between 10% and 90% = "pass"
17
18 Arithmetic mean value of data bytes is 127.8723 (127.5 = random).
19 // Close to the value of a perfectly random dataset
20
21
22 Monte Carlo value for Pi is 3.147572351 (error 0.19 percent).
23 // Small error margin = spatial distribution of bytes is highly uniform
24
25 Serial correlation coefficient is -0.001412 (totally uncorrelated = 0.0).
26 // The closer to 0, the less there is a dependence between each byte.
```

All the measured values are close to their ideal values in the case of total randomness. Once again, this provides no certain proof as to whether the construction used to transform the plaintext and ciphertext commands is truly random but is a further source of confidence.

4.2.3 Memory footprint

As the AIBO does not directly use a compiled binary, it was worth including it in the measures:

It is interesting to note that the difference in size between the XOR version and the secure version – which, by comparison, consists of far more files and lines of code (over 4,000 lines of code) – is only 10% larger as the added cryptographic code compiles compactly and the Memory Stick file is gzip-compressed.

Implementation	Packaged Size (TINYCONS.BIN)	Unpackaged Size (tinyConsole.bin)
XOR Prototype	165.1 KB (169,018 bytes)	675 KB (691,784 bytes)
Secure Final	193.8 KB (198,454 bytes)	747 KB (765,864 bytes)
<i>Overhead</i>	<i>+28.7 KB (29,436 bytes)</i>	<i>+72.0 KB (74,080 bytes)</i>
<i>Growth</i>	<i>+17.4%</i>	<i>+10.7%</i>

Table 4.1: Comparison of packaged .BIN and unpackaged .bin sizes and cryptographic overhead

Implementation	Files	Code Lines	Comment Lines	Total Lines
XOR Prototype	8	365	108	473
Secure TinyConsole	25	3,595	783	4,378
<i>Cryptographic Overhead</i>	<i>+17</i>	<i>+3,230</i>	<i>+675</i>	<i>+35</i>
<i>Relative Growth</i>	<i>+212.5%</i>	<i>+884.9%</i>	<i>+625.0%</i>	<i>+825.5%</i>

Table 4.2: Comparison of source code footprint and cryptographic overhead

4.2.4 Latency comparison

One final, and arguably most important factor, is the latency caused by the security measures added to the communication between the robot and the client. There are two distinct added costs: a one-time **connection setup** cost and a recurring **per-command** cost paid on every exchange.

The connection setup cost takes approximately 0.67s. In comparison, the base XOR connection setup takes about 20ms. However, this cost is irrelevant when considering the responsiveness of an already open session and is not considered further.

The per-command cost was measured by timing the RTT of a standard PING command across 2 000 samples for both versions (XOR baseline and secure TinyConsole). The reason for the choice of the PING command is that it does not require any additional computation time, as is the case with a movement command. This keeps the focus on overhead.

The results are displayed in the following table:

Metric	XOR baseline	ChaCha20-Poly1305	Δ
Samples	2000	2000	
Min (ms)	2.477	64.983	+62.506
Mean (ms)	5.576	77.456	+71.880
Median (ms)	3.583	75.860	+72.277
p95 (ms)	9.735	80.894	+71.159
p99 (ms)	16.770	95.085	+78.315
Max (ms)	236.093	595.241	+359.148
Std dev (ms)	14.106	19.271	+5.165

$$\Delta = \text{ChaCha20-Poly1305} - \text{XOR baseline}; \text{ mean overhead } +71.880 \text{ ms}$$

Table 4.3: Round-trip latency of one PING command over the TinyConsole link

This reveals a very significant difference and increase between the two versions. The XOR

baseline has a median RTT of 3.58 ms where secure AEAD implementation has a median of 75.86 ms and the mean overhead is +71.88ms ($\times 13.89$).

The overhead can be isolated as an additional 72 ms per command cycle. However, it must be noted that this value is not only the cost of ciphering the 4-byte PING command. In order to prevent traffic analysis, the secure TinyConsole additionally pads every payload to a strict 256-byte block.

Above all, this overhead cannot be attributed solely to the process of encrypting and authenticating each frame. ChaCha20+Poly1305 costs a few cycles per byte; even processing the padded 256-byte block in both directions, the RTT should be far lower. A client-side execution of the same primitive performs per-command encrypt-and-decrypt in the single-digit microseconds and even though the hardware between the client and the robot is not comparable, the difference in timing cannot be explained by hardware alone.

The packet capture 4.5 localises the delay: the client sends the request at $t=0$, the robot's reply arrives 85.7ms later and already carries the acknowledgement of the request (combined data and ACK segment). The robot therefore does not withhold an ACK before answering: it spends the whole interval receiving, processing and replying. As the frame also travels as a single segment, Nagle's algorithm can be ruled out and setting TCP_NODELAY on the client confirmed this since no beneficial effect was observed.

No.	Time	Time difference	Source	Destination	Protocol	Length	Info
1	0.00000000		192.168.1.242	192.168.1.124	TCP	328	16889 → 7777 [PSH, ACK] Seq=1 Ack=1 Win=64897 Len=274
2	0.085687000	0.085687000	192.168.1.124	192.168.1.242	TCP	328	7777 → 16889 [PSH, ACK] Seq=1 Ack=275 Win=8192 Len=274
3	0.141354800	0.055667800	192.168.1.242	192.168.1.124	TCP	54	16889 → 7777 [ACK] Seq=275 Ack=275 Win=64623 Len=0

Figure 4.5: Packet capture of a single PING exchange

The delay being solely internal to the robot, the object's own processing was investigated. Disabling the motor system (`OpenPrimitives` and `SetMotorPower` commented out, verified by the robot no longer moving when receiving messages) did not impact the reported values (median 75.63ms).

What did have a major impact was removing the allocation of the motor command-vector data (`NewCommandVectorData`) as it reduced the median from 75.86ms to 8.98ms (with a minimum of 3.39ms), similar to measurements of the XOR baseline:

Metric	ChaCha20-Poly1305	AEAD, network only	XOR baseline
Samples	2000	2000	2000
Min (ms)	64.983	3.397	2.477
Mean (ms)	77.456	16.216	5.576
Median (ms)	75.860	8.987	3.583
p95 (ms)	80.894	35.193	9.735
p99 (ms)	95.085	197.208	16.770
Max (ms)	595.241	254.131	236.093
Std dev (ms)	19.271	29.044	14.106

Table 4.4: Round-trip latency differences when the motor command-vector allocation is removed.

The explanation lies in the very essence of OPEN-R: the secure object registers a subject which is connected to the robot's effector, and an OPEN-R object processes only one

message at a time. Once that subject is active, the object is drawn into the effector’s ready/notify synchronisation and its network callbacks (`ReceiveCont`) are interleaved with, and thus paced by, the effector’s periodic frame cycle. Each reply is then emitted at a motor-frame boundary which is what the recorded RTT measured. Disabling motor power does not break this process, which is why only the removal of `NewCommandVectorData` eliminates this overhead.

Were this overhead considered a significant problem, it would come from the construction of the secure communication which bundles motor displacement and network handling in a single object, which does not benefit from OPEN-R modularity. This delay is not attributable to the cryptography (median of 8.987ms compared to the 3.583ms of the XOR baseline).

Removing this coupling is not feasible without designing and implementing an additional OPEN-R object.

The following histogram further illustrates the delay differences between the XOR baseline, secure TinyConsole and without motor actuation. RTTs are tightly clustered rather than spread. This also demonstrates that the added latency is highly consistent.

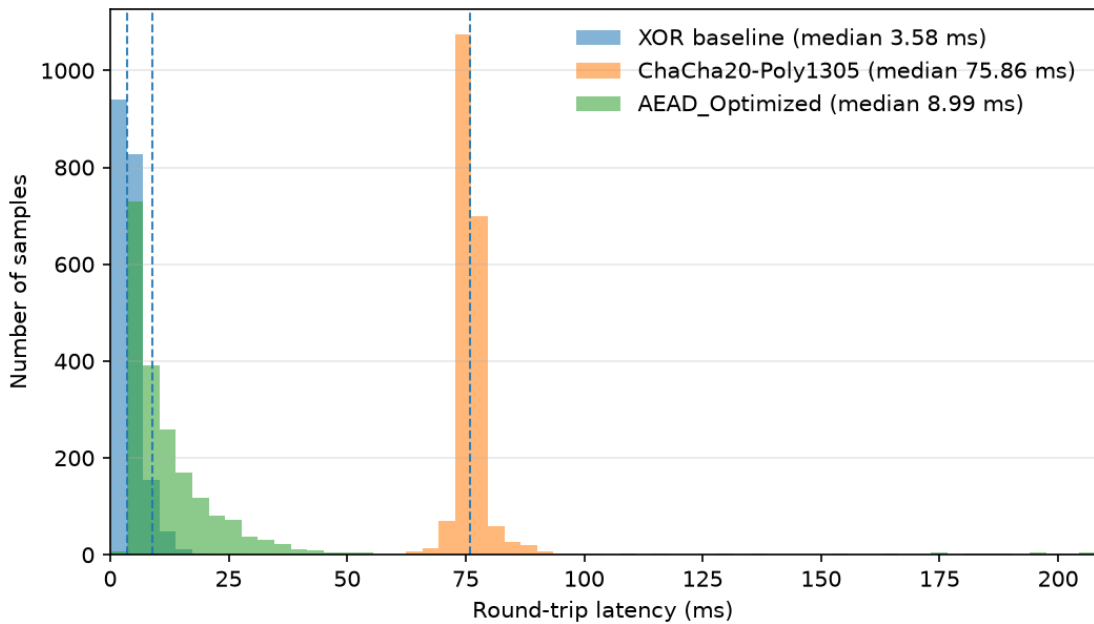


Figure 4.6: Histogram of the RTT of the three versions of TinyConsole

The bigger question is whether this would discernibly degrade operability. With a median RTT of around 76 ms, this translates to a one-way command latency of roughly 38 ms. A study from 1996 [53] measured that the reaction time for a human is around 150 ms which clearly indicates that the extra time taken by the secure version is not an issue in the context of remote control by a human operator. The occasional tail values are consistent with sporadic Wi-Fi jitter rather than execution failures, given the tight clustering of the distribution in the histogram.

Chapter 5

Discussion & Conclusion

The goal of this final chapter is to analyse and draw conclusions from the work that has been done in order to enhance the security of the AIBO ERS-7 in relation to the identified threats.

5.1 Comparison with state of the art and related work

As already mentioned in Section 2.2, there is no equivalent research that solely focus on securing legacy robots. No prior work was found that retrofits authenticated encryption onto a legacy robot using its original toolchain.

While the implementation is platform-specific (OPEN-R and Aperios), the method of defining a threat model, implementing an application-layer AEAD and endpoint authentication and finally measuring the resulting additional overhead could be ported to other legacy systems.

While the security analysis of the commercial Pepper robot [37] and the industrial ABB IRB140 [18] focused on finding and disclosing vulnerabilities, this work focuses on building and measuring mitigations.

It is indeed crucial and vital to highlight vulnerabilities in legacy systems, but if no solutions exist or are put in place, then the only option is to get rid of those systems (which has already been pointed out as, in some cases, not being an option, see Section 1.2). Furthermore, as mentioned earlier, the number of systems regarded as legacy is constantly increasing and if there is not a greater focus on finding solutions using the tools provided by the manufacturers, the situation will only get worse.

The research mentioned in Section 2.2.1 was the one that most closely resembled the work carried out on the ERS-7 but differences are numerous:

- The authors do not provide comparative data between the base system and the x-Ach architecture such as latency overhead, additional memory space filled or building blocks used (such as, as has already been pointed out, the SSH implementation used).
- In one case, the aim is to add middleware to enable legacy components to work with a modern environment; in the other, to enable secure communication between the robot and a remote client. While the work needed to make possible this secure communication required certain elements, such as movements, to be reimplemented, this was primarily done in order to demonstrate a functional and practical proof of concept that went beyond a simple exchange of data.
- It is difficult to make a meaningful comparison between this research and a hypothetical alternative modernisation of the robot, as it lacks concrete evidence while the work to secure the ERS-7's network communications is clearly specified, tested and measured.

A comparison can be drawn with the research from Kertész [8] in that, by using his improvement made to the original toolchain and the compiler flags, it should be possible to further reduce the size of the compiled binaries. The design choice taken with TinyConsole was to propose an implementation that must work on unmodified OPEN-R SDK versions and can therefore be replicated universally by anyone familiar with the OPEN-R SDK (which is still available on websites such as DogsBody).

In Section 2.2.2, multiple retrofit strategies were proposed such as air-gapping the insecure network used to establish a communication between the robot and the remote client. The proposed TinyConsole ensures that this communication is secure while leaving open the possibility of using some of those strategies such as air-gapping the network or using an Endpoint Detection and Response (EDR) agent on the client’s computer to monitor the data sent by the robot and create logs to ensure non-repudiation.

As regards the implementation of the symmetric encryption using ChaCha20 with Poly1305 for MAC, the measured additional latency overhead by comparing the XOR version of TinyConsole with the version where movements are disabled was 5.4ms (median). This is a negligible overhead and using another more-optimised cipher like the AEAD construct from Ascon would not have significantly changed the result.

Although, as stated, the actual median RTT is 75 ms, this is mainly due to the fact that the TinyConsole should have been split into two objects, rather than handling the motion and cryptographic components within a single one.

Finally, an aspect was not found in related research on legacy systems: the use of a persistent-counter replay mitigation for an entropy-less device. Although this solution is far from ideal, as it chooses to treat the risk of physical access to the Memory Stick as being outside the scope, it has the advantage of significantly reducing the risk of a nonce being reused.

This is an aspect of this work that could be reused on other legacy systems where a non-volatile memory exists but there is no source of entropy or RNG.

5.2 Lessons learned: How to secure legacy systems?

Securing legacy systems or rather, reducing their vulnerabilities, is no easy task and the work done on the ERS-7 is further proof of this.

The development experience is not exactly the most comfortable:

- Using a less-than-ideal development environment in order to be able to use the original toolchain (Ubuntu 8.04 LTS).
- Physical (limited RAM, memory storage, old CPU) and software limitations (lack of encryption on stored data, no support for newer versions of C++ or libraries) are numerous and further complicate the task.
- Going through the platform-specific development process can be very tedious and time-consuming, here for the AIBO: implement objects in C++, compile them into binaries using the toolchain, copy them onto the physical Memory Stick by combining them with one of the OPEN-R configurations (BASIC, WLAN or WCONSOLE), put

the Memory Stick into the robot and boot it and only then using the telnet console (if enabled) is it possible to know if there are bugs in the implementation.

- The documentation and code samples can be limited depending on the platform and even traditional Large Language Model (LLM) tools are sometimes not trained in such a way that they can be used to simplify the development process.

In order to secure a legacy system, a broad methodology needs to be followed in order to increase its chances of success:

1. Define what can be modified or replaced on the system and what cannot. This will help to narrow things down and focus on what is feasible. On some platforms it will not be possible to change the network endpoint; on others, there will be no development SDK available; and on others still, there will be no means of storing data persistently. For the ERS-7, this corresponds to the WEP encryption used by the 802.11b radio that cannot be patched. As there are no means to modify the LAN card's firmware (as the underlying Atheros OS is inaccessible), and that replacing the card is out of the question (a newer network card would not be recognised by the OS), this introduces a significant limitation: in order to secure the network link, it is necessary to work at the application layer of the network.
2. Define a threat model and categories of threats that need to be mitigated by order of urgency and potential impact. Given that we are already aware of the limitations mentioned in the previous point, this gives us a better understanding of what needs to be done and what is actually possible. In the case of the AIBO, it took the shape of a STRIDE and risk matrix. Threats were separated into three categories and focus was put on the high-priority threats.
3. Choose mitigations that match the platform's constraints. For the ERS-7, as there was no on-board entropy, this meant that it was not possible to use Ephemeral Diffie-Hellman (DH) to generate temporary private keys to cipher each new connection. This therefore directly implies that forward secrecy is not possible and that in order to have a system that is resistant to replay attacks, it is necessary to have persistent state (static nonce stored on the robot's Memory Stick which is used to generate the first nonce when establishing a new connection and TinyConsole will refuse to continue the communication when the counter cannot advance or keys are missing). Thus, deterministic signatures had to be used to ensure authentication and symmetric cryptography was preferred. It is also important to bear in mind that these mitigation measures must not reduce the system's capacity; otherwise this would be tantamount to taking a step backwards. The AIBO robot can still operate at full capacity and all its sensors are still accessible. In practice, objects must be built around the TinyConsole implementation in order to ensure that all data transmitted from the robot is secure.
4. Chosen mitigations should be implemented using standardised, recognised and reliable solutions. There is no point in deciding to encrypt network communications if it means manually re-implementing an encryption algorithm, as this would result in introducing unnecessary risks. Similarly, it is important to remain realistic when choosing these implementations and to stay within the constraints of the legacy system. TweetNaCl and Will Glozer's implementation were therefore chosen. Glozer builds

on the Bernstein’s original implementations of ChaCha20 and Poly1305; TweetNaCl on the original implementation of Ed25519. Although the Ascon implementation was also recognised by NIST as a standard, it was not selected for the reasons set out in section 3.5.3, but also because the client-side implementations were not sufficient.

5. Finally, it is crucial to test and monitor the system thoroughly once these mitigation measures have been put in place. An implementation or configuration error can easily occur and could undermine all the work that has been done. In this sort of limited system, it is also important to know – even if it has been theorised – what the actual cost is in terms of latency, storage space, CPU and RAM usage, etc. The more metrics are known and the fewer unknowns there are, the more secure the system can be considered to be. It is far better to be aware of a risk that, whilst not mitigated, is being monitored or accepted, than to be unaware of it. This is covered in detail in Chapter 4 and helps to clarify an element that has already been mentioned several times: due to the way Aperios works, sending movement commands requires using an object that receives these commands via its observer and processes them only one at a time. This therefore results in a significant delay caused by the platform and not the cryptography (additional 66.873ms when calculating the median).

5.2.1 Taking it a step further: how can a system be made future-proof?

As discussed in Section 1.2, legacy systems are more often than not systems that were designed to have a long lifespan (PLC, electric grids, etc.).

In Section 2.2.2, possibilities of securing legacy systems and measures put into place to future-proof a system were also discussed.

However, those cannot really be considered a sound methodology. When discussing long-term (in some cases, very long term) security, it is vital to mention the sustainability of both security and safety (two important aspects where, as already discussed, ‘safety’ is sometimes preferred to ‘security’, as was the case with the IRB140 industrial robot [18]).

In a 2019 paper by Pavard et al. [54], multiple architectural requirements and design principles for achieving sustainable security and safety are proposed, that is to say "*a system able to maintain its security and safety, but desirably also its functionality for at least its design lifetime*".

Although this is not the main focus of this work, it is interesting to draw parallels between these elements and the work carried out.

Those requirements and design principles were:

- Well-defined components and isolation: OPEN-R is inherently modular and Tiny-Console runs as a single object in user mode with memory protection enabled, isolated from other objects by the message-passing model. However, network handling and motor actuation were placed in the same object which departs from the principle of isolation between the two functions. Furthermore, as the OS is proprietary and not accessible, it is not possible to verify whether the object is fully isolated at the memory level or whether any vulnerabilities exist that could compromise this isolation.
- Avoiding single points of failure: The ChaCha20 key and the Ed25519 identities both protect the network communication. If the symmetric key was to be compromised, it

would not allow an attacker to impersonate either endpoint, however confidentiality relies entirely on this secret. In the event of a data breach, its disclosure would expose all past and future communications as there is no forward secrecy.

- Having multiple lines of defence: TinyConsole implements several independent controls with each addressing a different threat:
 - Mutual Ed25519 authentication guards the session before any encrypted frame is processed.
 - A Poly1305 tag authenticates every frame.
 - A persistent monotonic counter blocks replay across reboots.
 - A direction bit in the nonce prevents collision between the two transmission directions.
 - Fixed-size padding limits traffic analysis.
 - The object fails closed when an element or key is absent; or when the counter cannot advance.
- Making use of long-term secrets and confidentiality: While long-term Ed25519 identities are exchanged outside of the communication, which is architecturally correct (exchanging the public keys over the link would not prove the identity of the key's holder), no protected storage exists. The secrets are compiled into the binary and, as accepted in the risk analysis, physical access to the Memory Stick exposes them.
- Using robust input handling: The receive path is a byte-aligned state machine that reads exact lengths for each handshake phase and rejects anything else. Frames are validated against the expected fixed block size before decryption, the inner length field is bounds-checked after decryption, and no plaintext is acted upon until the Poly1305 tag has been verified. Commands are matched against an explicit list of accepted items rather than parsed.
- Contain subsystem failure: Failures are handled defensively:
 - A link error force-closes the communication and leaves reconnection to a timer.
 - A failed signature verification terminates the connection.
 - The motion state machine falls back to an idle state if the effector observer is not ready.

However, the containment boundary is limited as in the case of a failure either in networking, actuation or encryption/decryption, it would affect all three rather than a specific object.

- Applying replication to tolerate subsystem failure: The work was only carried out on a single robot, a single operator and a single network link with no redundancy at any level. The reconnection logic in the ROS 2 bridge provides recovery, not replication. Nevertheless, this principle targets systems in which redundant hardware is economically justifiable, which does not apply to this specific case.
- Diversifying nodes and components: There is only one robot and one control program which does not make node diversity attainable. The two endpoints do run indepen-

dent implementations of the same primitives (TweetNaCl and the ChaCha20's C implementation on the robot and the `cryptography` package on the client) and this cross-implementation agreement can be seen as a correctness check throughout development. This was also one of the reasons why the Ascon's AEAD implementation was rejected as no comparably hardened and tested client-side implementation was available.

- Being able to adapt and reconfigure the system: This is the principle for which the robot and implementation diverge the most. There is no easy way to directly update the program, no cryptographic agility and no runtime configuration of security parameters. Everything is fixed at build time and the keys are compiled into the binary which results in needing to use the toolchain to recompile and reflash an updated version of the program on the Memory Stick. Only non-sensitive parameters such as the robot's address in the ROS 2 launch file or Python's client can be altered without rebuilding.
- Minimising assumptions: The assumptions are explicit and documented: an isolated wireless network, a single operator, out-of-band key distribution, physical security of the Memory Stick is taken for granted, and the correctness of third-party cryptographic implementations that were adopted on provenance rather than audited. The principle asks for a minimum number of them and several of them are strong. Yet, most cannot be diminished without capabilities the AIBO does not offer.
- Prioritising having a simple and verifiable core system: TweetNaCl was chosen precisely as it was a minimal, single-file, dependency-free implementation written by the designers of the algorithm it implements and no cryptographic algorithm was re-implemented for this work. The protocol itself is a small state machine with a fixed frame format.

Two observations can be drawn from this comparison.

First, the design principles that this work satisfies are those concerning the construction of the security layer (input handling, defence in depth, a small trusted core) because these depend on design discipline rather than on platform capability.

Second, the requirements that cannot be satisfied (reconfigurability, replication, protected long-term storage) all presuppose capabilities that the robot and OPEN-R (Aperios) do not provide.

This is arguably the most useful learning to take away for systems being designed today with a long expected lifespan: sustainable security and safety is a property that must be provided from the outset and can hardly be retrofitted. In the specific case of the ERS-7, this comes mainly in the shape of an accessible source of entropy.

5.3 Limitations of validity

The proposed methodology is deliberately broad so that it can be applied to a wide range of legacy systems. As already discussed in Section 1, the term 'legacy system' encompasses a vast number of different systems – too many to be able to propose a precise methodology tailored to each individual case.

Thus, a key limitation of the methodology is that, by its very nature, it is not 100% suitable for all types of legacy systems.

Other limitations have already been discussed throughout this document but are worth recalling here:

- Limitations of Aperios and of the hardware of the ERS-7 mean that there is no forward secrecy. If the symmetric key used to cipher communication between the robot and the remote client was to be leaked, then all previous and future communications could be deciphered.
- Similarly, the symmetric key used is static and directly embedded into the code, and by extension the binary, of the OPEN-R program. It is therefore possible to retrieve the key as soon as access is granted to either the source code or the compiled binary.
- None of the medium- and low-priority threats are mitigated even if they had already been regarded as planned for or accepted. Primarily, the risks associated with physical access to the robot's Memory Stick – which are the risks most likely to result in the leakage of cryptographic keys – are accepted, given that no acceptable mitigation measures were possible.
- The risk of a DoS attack (DF5) is also still present, and although it was considered 'planned', during the testing phase it was clearly demonstrated, using an attack scenario, that it was possible to disconnect the robot and the client. The attack execution is now well characterised which makes future work on a mitigation easier by giving a concrete starting point.
- The use case scenario of TinyConsole is with one robot communicating with one remote operator over one network link. While this is not a limitation by itself given that the robot is designed to operate with one remote operator at a time due to its hardware limitations, it is not possible to have multiple clients registered in TinyConsole in order to have multiple remote control points or multiple networks on which to use the robot.
- The **ent** test suite was only applied on a small sample which limits the usefulness of its results. To obtain results that could be fully trusted, several megabytes of data would be required. This would also make it possible to compare the results by running the sample through NIST STS or Dieharder.
- Similarly, the audit of the nonces was carried out on only 17 sessions, which does not provide any certainty that the nonce generation implementation is entirely correct. 17 sessions cannot reveal a hidden implementation issue, and exhaustive sampling is infeasible; the correctness of the implementation thus rests on the argument that the persistent counter cannot repeat with the audit serving only as a consistency verification.
- Neither the OPEN-R program nor the remote client were formally verified.
- The cryptographic algorithm implementation were directly adopted and so confidence in their correctness depends on their authors. While they were checked, they were mainly selected based on their reputation and provenance.

5.4 Future work

Future work can be divided into two categories: work to continue improving and adapting TinyConsole and work based on the methodology adapted here and applied to other legacy systems.

5.4.1 Sequels to TinyConsole

First of all, it would be interesting to see whether TinyConsole could be adapted to the ERS-2X0 series of AIBO without too much additional work which, although they do not initially have LAN capability, can be upgraded by inserting a LAN card that was sold separately. In theory, the way in which data is received and sent via the ANT would need to be adapted.

Then, if focusing solely on the ERS-7, multiple enhancements to TinyConsole are possible:

- A priority addition would be to split TinyConsole into two objects: one responsible for receiving and transmitting data via its TCP port, and the other responsible for the robot's physical movements. This would significantly reduce RTT latency.
- Medium-priority threat DF3 would need to be mitigated using a system for storing persistent logs on the Memory Stick whilst taking into account its limited storage space (ideally by encrypting these logs using the robot's private key).
- Medium-priority threat DF5 should be solved by finding a way to ignore TCP RST packets that do not originate from the remote client and therefore find a way to identify the source of a received packet before processing it.
- As already conceptualised, sending the encrypted video stream from the robot's camera to the remote client which should therefore also be capable of displaying the streams in real time and/or storing them. This could be done using UDP by sending the encrypted data from TinyConsole to another OPEN-R object, which would be responsible for sending the packets via UDP (as TinyConsole uses TCP).
- Another useful addition – which actually encompasses several – would be to introduce more possible movements for the robot and provide finer control over its movements such as left and right turn, choosing the orientation of the robot's head and therefore of its camera, to enable it to bend down without lying down, etc.
- As the robot is also fitted with microphones, it could be worth trying to transmit the captured audio stream to the remote controller via TinyConsole.
- Finally, regarding low-priority threats, while it seems less likely that possibilities to mitigate those threats exist, it would be worth investigating if there is any hardware-backed storage path that exists.

Cryptographic improvements

Multiple cryptographic improvements are possible:

- Design and use an entropy source that is sufficiently robust to serve as an input for an RNG such as sensor noise (accelerometer, infrared, microphone, CMOS camera) or timing jitter. This foundation could then be used to implement a DH key exchange by generating pairs of private and public keys for each new connection.

- Based on the previous point, forward secrecy could be done via ephemeral X25519.
- Post-quantum migration should target the asymmetric part of the cryptographic scheme in priority. ChaCha20+Poly1305 with a 256-bit key retains roughly 128-bit security against Grover’s algorithm. In comparison, Ed25519 is broken directly by Shor’s algorithm. The migration path is thus to replace the handshake signature scheme rather than to alter the cipher.
- Additional security measures targeted at ROS 2 with SROS 2 (that is built on top of DDS-Security) and thus going beyond the localhost-only discovery limitation.
- Formal verification of the handshake state machine.
- Broader measurement using multi-environment latency, a multi-megabyte **ent** sample and more sessions for the nonce audit.
- Key management and rotation also would need dedicated attention. In the current implementation, the ChaCha20 and Ed25519 keys are compiled into the OPEN-R binary and a separate key file would offer no additional security as it would be readable in plaintext. The underlying consequence is that rotating any key requires recompiling the object, and physically putting it into the Memory Stick. There is no way to revoke a compromised client key without physical access to the robot. A key-derivation function could expand a single long-term secret into per-purpose subkeys in order for a stream key to be used without needing to regenerate the symmetric key. Additionally, a trust-on-first-use (TOFU) model in which the client records the key presented at the first connection rather and warns on any subsequent change could be considered an improvement (rather than manually storing the public key). However, this would have the disadvantage of not being able to verify the identity of the endpoint that generated this key.

5.4.2 Applying the methodology to other legacy systems

As regards the methodology, it would be interesting to apply it to as many different types of legacy systems as possible to verify and gather data about the feasibility of enhancing the security of such systems.

Examples are multiple: industrial controllers on serial/fieldbus links, legacy medical devices such as MRI scanners, building automation or industrial robots. A useful distinction is between systems whose endpoints are wired, where a gateway remains architecturally possible, and those whose endpoints are wireless, where it is not — as is the case with the ERS-7.

Given their large number and wide variety, the more this can be done, the more feasible it would be to implement specific techniques tailored to the different types of systems and manufacturers.

5.5 Conclusions

To conclude this master thesis, let us summarise the original problem which was whether legacy systems, which have a tendency to remain in service long after their vendor support

has ended and their security measures are no longer effective, could still be secured with regard to their network connection.

This question was investigated using the AIBO ERS-7 from Sony, a robot that went on sale publicly over 20 years ago and had been widely sold and used in the research field thanks to its advanced capabilities for the time. This robot had LAN capacity thanks to its 802.11b wireless radio which only supported WEP which could not be patched and whose OS is inaccessible.

The question, therefore, was whether it was possible, using only the tools originally provided by the vendor (in this case, the OPEN-R SDK) to enhance the security of its network connection.

On that basis was built TinyConsole: an OPEN-R object providing a ChaCha20+Poly1305 AEAD channel with mutual Ed25519 authentication and a persistent anti-replay counter with the addition of a Python client (reflecting the OPEN-R construction) and a ROS 2 bridge - all compiled with the original SDK unmodified.

Multiple research questions had been laid out:

1. *“Can confidentiality and integrity be provided at application level within the ERS-7’s limitations?”*

A working implementation ensuring both confidentiality (ChaCha20) and integrity (Poly1305) provided at application level was established within the ERS-7 limitation. The integrity was demonstrated by the tamper test where a single flipped bit in ciphertext or tag caused a rejection and connection teardown (4.2). Ciphertext showed no exploitable structure under **ent** (Section 4.2.2).

2. *“What is the cost of the retrofit in latency and footprint, and does it remain usable for real-time teleoperation?”*

The cost was measured with a median RTT latency of 75.86ms versus 3.58ms for the XOR baseline, an additional footprint of 28.7 KB when packaged (+17.4%), 72 KB when unpackaged (+10.7%) and more than 4,000 lines of code. However, the real source of the additional latency was not from the cryptographic overhead –the AEAD scheme adds roughly 5ms once the motor pipeline is decoupled, of which the cipher itself accounts for microseconds– but from the OPEN-R’s single-message-at-a-time coupling between the network and motor objects. But even with an RTT of 75 ms, this is still well within acceptable limits (below the 150ms human reaction time [53]).

3. *“Can the endpoints be mutually authenticated and what residual risks remain?”*

Yes, both endpoints sign a shared transcript which is verified by the MITM key-substitution test where a tampered robot signature causes the client to refuse the connection (Figure 4.3). What can be considered residual is that there is no forward secrecy (static pre-shared key) and the replay resistance depends on the Memory Stick counter.

4. *“What is the impact of an attacker gaining physical access in terms of security?”*

The impact is still severe and characterised rather than mitigated. Access to the Memory Stick yields the keys and permits rolling back the anti-replay counter. The Memory Stick offers no secure element which resulted in accepting these threats as out of scope (DS1 and DS3).

5. “*Can the resulting secure remote control be interfaced with modern controllers using ROS 2’s middleware?*”

Yes, a Python bridge on Jazzy exposes the robot through `/cmd_vel` and topic-based commands, with the cryptographic layer untouched as the node instantiates the same authenticated client. DDS discovery is constrained to localhost so the middleware does not widen the attack surface (3.6.2). This therefore allows a user to control the robot using a keyboard or any other modern controller via ROS 2 without modifying its OPEN-R program.

More generally, this can be considered the first known security analysis on the AIBO ERS-7, the first AEAD and Ed25519 retrofit within the original toolchain, the first persistent-counter replay mitigation for this entropy-less device, the first implementation of a ROS 2 bridge tailored for this robot and finally the first time a thorough assessment of the cost of adding these security measures has been carried out.

On constrained legacy platforms, the cost of retrofitting security is widely dominated by the platform’s architecture and its quirks and inherent difficulties rather than by the cryptography. What is more, the lack of entropy leads down a chain of considerations: no entropy means no ephemeral DH, no forward secrecy and thus the requirement for a persistent state. And it is indeed this limitation that is most likely to arise with legacy systems, and perhaps one of the most restrictive ones (without considering the potential lack of SDK and other development tools).

5.5.1 Open-source

Before reaching a final conclusion, it is important to point out that this work should – and, above all, the code and documentation produced as a result – be made open-source and available via the public GitHub repository https://github.com/Gaschrod/AIBOSecure_Console.

It seems unlikely that it would be used for commercial purposes, but having seen just how much, even 20 years on, this robot can still stir up passion within the community of enthusiasts, there is hope it can be developed further by people with a genuine interest in it.

5.5.2 The Man-Bites-Dog Phenomenon

When a system is considered obsolete, the default reflex is to replace or isolate it (smartphones, laptops, servers, computers, automation machinery and robots). Unfortunately or fortunately, sometimes that is not possible.

In those situations, a precise retrofit using only the manufacturer’s own tools is feasible, measurable and – on this evidence – imperceptible to the user.

When a legacy system is decommissioned because it is no longer secure, that is a story, but when a legacy system’s security is enhanced using only its legacy toolchain, that is a *good* story.

Bibliography

- [1] Dykstra, Metcalf, L., Josiah, Spafford, E.H.: Cybersecurity Myths and Misconceptions: Avoiding the Hazards and Pitfalls that Derail Us (2023) (Cited on page III.)
- [2] Roty, A., Mühlberg, J.T., Determe, J.F.: Opportunities and Challenges in Securely Reusing and Repurposing Mobile Devices (Jun 2026), <https://arxiv.org/abs/2606.06181v2> (Cited on page III.)
- [3] Mees, W.: Pragmatic Cybersecurity: A Manager's and Architect's Guide to Security Architecture, Risk Management, and Cyber Resilience. Independently published (2020) (Cited on pages VI and 28.)
- [4] SAA Dictionary: legacy system, <https://dictionary.archivists.org/entry/legacy-system.html> (Cited on page 1.)
- [5] Williams, J.L.: The Stolen Story and Other Newspaper Stories. C. Scribner's Sons (1899), google-Books-ID: QkkLAAAAIAAJ (Cited on page 1.)
- [6] Sony Global - Q3 FY2005 Sony Group Earnings Announcement (May 2013), <https://web.archive.org/web/20130516161052/http://www.sony.net/SonyInfo/IR/info/presen/05q3/qfhh7c000008adfe.html> (Cited on page 1.)
- [7] Entertainment Robot "aibo" Announced, <https://www.sony.com/en/SonyInfo/News/Press/201711/17-105E/index.html> (Cited on page 2.)
- [8] Kertész, C.: Improvements in the Native Development Environment for Sony AIBO. International Journal of Interactive Multimedia and Artificial Intelligence 2(3), 50–54 (Sep 2013) (Cited on pages 2, 4, 13, 20, and 61.)
- [9] OPEN-R Architecture Specifications To Be Made Public, <https://www.sony.com/en/SonyInfo/News/Press/200205/02-017E/index.html> (Cited on page 2.)
- [10] Knight, W.: Aibo custom code pulled from website, <https://www.newscientist.com/article/dn1495-aibo-custom-code-pulled-from-website/> (Cited on page 2.)
- [11] Yokote, Y.: The Apertos reflective operating system: the concept and its implementation. ACM SIGPLAN Notices 27(10), 414–434 (Oct 1992) (Cited on page 2.)
- [12] Case, B.: RM7000 strong on cost/performance. Microprocessor Report 10(14), 36–39 (1996) (Cited on page 2.)
- [13] Sweetman, D.: See MIPS run. Elsevier (2010) (Cited on page 2.)
- [14] Kimura, R., Miura, K., Murata, H., Yokoyama, A., Naganuma, M.: Consideration of physiological effect of robot assisted activity on dementia elderly by electroencephalogram (EEG): Estimation of positive effect of RAA by neuroactivity diagram. In: Proceedings of SICE Annual Conference 2010. pp. 1418–1422 (Aug 2010) (Cited on page 3.)

- [15] Banks, M.R., Willoughby, L.M., Banks, W.A.: Animal-Assisted Therapy and Loneliness in Nursing Homes: Use of Robotic versus Living Dogs. *Journal of the American Medical Directors Association* 9(3), 173–177 (Mar 2008) (Cited on page 3.)
- [16] Ode to AIBO | Humans To Robots Laboratory, <https://h2r.cs.brown.edu/ode-to-aibo/> (Cited on page 3.)
- [17] Report | Telehealth take-up: the risks and opportunities (Kaspersky) (Dec 2021), <https://www.benton.org/content/report-telehealth-take-risks-and-opportunities-kaspersky> (Cited on page 4.)
- [18] Quarta, D., Pogliani, M., Polino, M., Maggi, F., Zanchettin, A.M., Zanero, S.: An experimental security analysis of an industrial robot controller. In: 2017 IEEE Symposium on Security and Privacy (SP). pp. 268–286 (2017) (Cited on pages 4, 15, 60, and 63.)
- [19] Average age of the EU vehicle fleet, by country (Sep 2024), <https://www.acea.auto/figure/average-age-of-eu-vehicle-fleet-by-country/> (Cited on page 4.)
- [20] Is Your SCADA System Aging? Maybe It’s Time to Take Stock. (2019), <https://www.tpomag.com/editorial/2019/12/is-your-scada-system-aging-maybe-its-time-to-take-stock> (Cited on page 4.)
- [21] Karastoyanov, D., Karastanev, S.: Reuse of Industrial Robots. *IFAC-PapersOnLine* 51(30), 44–47 (2018) (Cited on page 4.)
- [22] Yang, L., Zhang, C., Xiong, Y., Wen, B., Xu, J.: Lifetime Prediction and Interpretability Analysis of Power Transformers Based on Multi-Model Feature Selection and Stacking Ensemble. *Mathematics* 14(13), 2417 (Jan 2026) (Cited on page 4.)
- [23] Alsabbagh, W., Langendörfer, P.: Security of Programmable Logic Controllers and Related Systems: Today and Tomorrow. *IEEE Open Journal of the Industrial Electronics Society* 4, 659–693 (2023) (Cited on pages 4, 14, and 15.)
- [24] Baillie, J.C.: Urbi: towards a universal robotic low-level programming language. In: 2005 IEEE/RSJ International Conference on Intelligent Robots and Systems. pp. 820–825 (2005) (Cited on page 5.)
- [25] Hohl, L., Tellez, R., Michel, O., Ijspeert, A.J.: Aibo and Webots: Simulation, wireless remote control and controller transfer. *Robotics and Autonomous Systems* 54(6), 472–485 (Jun 2006) (Cited on page 12.)
- [26] Zaratti, M., Fratarcangeli, M., Iocchi, L.: A 3D Simulator of Multiple Legged Robots Based on USARSim. In: Lakemeyer, G., Sklar, E., Sorrenti, D.G., Takahashi, T. (eds.) *RoboCup 2006: Robot Soccer World Cup X*. pp. 13–24. Springer, Berlin, Heidelberg (2007) (Cited on page 12.)
- [27] Saggar, M., D’Silva, T., Kohl, N., Stone, P.: Autonomous Learning of Stable Quadruped Locomotion. In: Lakemeyer, G., Sklar, E., Sorrenti, D.G., Takahashi, T. (eds.) *RoboCup 2006: Robot Soccer World Cup X*. pp. 98–109. Springer, Berlin, Heidelberg (2007) (Cited on pages 12 and 38.)
- [28] Röfer, T.: Evolutionary Gait-Optimization Using a Fitness Function Based on Proprioception. In: Nardi, D., Riedmiller, M., Sammut, C., Santos-Victor, J. (eds.) *RoboCup*

- 2004: Robot Soccer World Cup VIII. pp. 310–322. Springer, Berlin, Heidelberg (2005) (Cited on page 12.)
- [29] Kertész, C.: Rigidity-Based Surface Recognition for a Domestic Legged Robot. *IEEE Robotics and Automation Letters* 1(1), 309–315 (Jan 2016) (Cited on page 12.)
 - [30] Herrero-Pérez, D., Martínez-Barberá, H., LeBlanc, K., Saffiotti, A.: Fuzzy uncertainty modeling for grid based localization of mobile robots. *International Journal of Approximate Reasoning* 51(8), 912–932 (Oct 2010) (Cited on page 12.)
 - [31] Lakemeyer, G., Sklar, E., Sorrenti, D.G., Takahashi, T. (eds.): *RoboCup 2006: Robot Soccer World Cup X*, *Lecture Notes in Computer Science*, vol. 4434. Springer, Berlin, Heidelberg (2007) (Cited on page 12.)
 - [32] Stronger, D., Stone, P.: Selective Visual Attention for Object Detection on a Legged Robot. In: Lakemeyer, G., Sklar, E., Sorrenti, D.G., Takahashi, T. (eds.) *RoboCup 2006: Robot Soccer World Cup X*. pp. 158–170. Springer, Berlin, Heidelberg (2007) (Cited on page 12.)
 - [33] Nicklin, S.P., Fisher, R.D., Middleton, R.H.: Rolling Shutter Image Compensation. In: Lakemeyer, G., Sklar, E., Sorrenti, D.G., Takahashi, T. (eds.) *RoboCup 2006: Robot Soccer World Cup X*. pp. 402–409. Springer, Berlin, Heidelberg (2007) (Cited on page 12.)
 - [34] Hausknecht, M., Stone, P.: Learning Powerful Kicks on the Aibo ERS-7: The Quest for a Striker. In: Ruiz-del Solar, J., Chown, E., Plöger, P.G. (eds.) *RoboCup 2010: Robot Soccer World Cup XIV*. pp. 254–265. Springer, Berlin, Heidelberg (2011) (Cited on page 12.)
 - [35] Fidelman, P., Stone, P.: Learning Ball Acquisition on a Physical Robot. *International Symposium on Robotics and Automation (ISRA)* (2004) (Cited on page 12.)
 - [36] Lofaro, D.M., Ward, C., Bugajska, M., Sofge, D.: Extending the life of legacy robots: MDS-Ach, a real-time, process based, networked, secure middleware based on the x-Ach methodology. In: *2018 IEEE 15th International Workshop on Advanced Motion Control (AMC)*. pp. 72–77 (Mar 2018), iSSN: 1943-6580 (Cited on page 13.)
 - [37] Giaretta, A., Donno, M.D., Dragoni, N.: Adding Salt to Pepper: A Structured Security Assessment over a Humanoid Robot. In: *Proceedings of the 13th International Conference on Availability, Reliability and Security*. pp. 1–8 (Aug 2018), arXiv:1805.04101 [cs.CR] (Cited on pages 14 and 60.)
 - [38] Research, A.: Growing confidence in Robotics Led to US\$2.7 Billion in VC Funding in 2017, With Appetite for More (2018), <https://www.prnewswire.com/news-releases/growing-confidence-in-robotics-led-to-us2-7-billion-in-vc-funding-in-2017--with-appetite-for-more-300671853.html> (Cited on page 14.)
 - [39] Azevedo, M.A.: Sector Snapshot: Robotics Startups On Fire As Venture Funding Surges To Record Numbers In 2026 (Jun 2026), <https://news.crunchbase.com/robotics/startup-venture-funding-surges-2026-data/> (Cited on page 14.)
 - [40] Limin, J., Kang, E., Timperley, C., Scheffler, S.: Securing the Future of Robotics and Autonomous Systems | Carnegie Mellon University (Jul 2025), <https://www.cmu.edu>

`u/work-that-matters/energy-innovation/securing-future-robotics-and-autonomous-systems` (Cited on page 14.)

- [41] Guri, M.: SmartAttack: Air-gap attack via smartwatches. arXiv preprint arXiv:2506.08866 (2025) (Cited on page 15.)
- [42] Salama, D.D., Abd elkader, H., Hadhoud, M.M.: Performance Evaluation of Symmetric Encryption Algorithms. Communications of the IBIMA 10 (Jan 2009) (Cited on page 16.)
- [43] Ragnarsson, K.O., Mallett, J.: Performance Testing of ChaCha20-Poly1305 for Internet of Things and Industrial Control System Devices (Apr 2026), <http://arxiv.org/abs/2603.19150>, arXiv:2603.19150 [cs.CR] (Cited on page 16.)
- [44] Bernstein, D.J.: The poly1305-aes message-authentication code. In: International workshop on fast software encryption. pp. 32–49. Springer (2005) (Cited on page 16.)
- [45] Fluhrer, S., Mantin, I., Shamir, A.: Weaknesses in the key scheduling algorithm of rc4. In: Vaudenay, S., Youssef, A.M. (eds.) Selected Areas in Cryptography. pp. 1–24. Springer Berlin Heidelberg, Berlin, Heidelberg (2001) (Cited on pages 17 and 18.)
- [46] Serra, F., Baillie, J.C.: Aibo programming using OPEN-R SDK tutorial (2003) (Cited on page 29.)
- [47] Turan, M.S., McKay, K.A., Chang, D., Kang, J., Kelsey, J.: Ascon-based lightweight cryptography standards for constrained devices. In: NIST Special Publication (SP) NIST SP 800–232 ipd. National Institute of Standards and Technology (2024) (Cited on page 32.)
- [48] Chouhan, V., Aldarwbi, M., Sadeghi, S., Ghorbani, A., Chow, A., Burko, R.: Assessing the quantum readiness of cryptographic standards: Recommendations toward quantum-era compliance. Computer Standards & Interfaces 97, 104114 (Apr 2026) (Cited on pages 33, 34, and 35.)
- [49] Bathe, B., Anand, R., Dutta, S.: Evaluation of Grover’s algorithm toward quantum cryptanalysis on ChaCha. Quantum Information Processing 20(12), 394 (Nov 2021) (Cited on page 34.)
- [50] Bernstein, D.J., Duif, N., Lange, T., Schwabe, P., Yang, B.Y.: High-speed high-security signatures. Journal of Cryptographic Engineering 2(2), 77–89 (Sep 2012) (Cited on page 34.)
- [51] Bernstein, D.J., Lange, T., Schwabe, P.: The Security Impact of a New Cryptographic Library. In: Hevia, A., Neven, G. (eds.) Progress in Cryptology – LATINCRYPT 2012. pp. 159–176. Springer, Berlin, Heidelberg (2012) (Cited on page 34.)
- [52] Bernstein, D.J., van Gastel, B., Janssen, W., Lange, T., Schwabe, P., Smetsters, S.: TweetNaCl: A Crypto Library in 100 Tweets. In: Aranha, D.F., Menezes, A. (eds.) Progress in Cryptology - LATINCRYPT 2014. pp. 64–83. Springer International Publishing, Cham (2015) (Cited on page 34.)
- [53] Thorpe, S., Fize, D., Marlot, C.: Speed of processing in the human visual system. Nature 381(6582), 520–522 (Jun 1996) (Cited on pages 59 and 69.)

- [54] Paverd, A., Völz, M., Brasser, F., Schunter, M., Asokan, N., Sadeghi, A.R., Esteves-Veríssimo, P., Steininger, A., Holz, T.: Sustainable Security & Safety: Challenges and Opportunities. In: Asplund, M., Paulitsch, M. (eds.) 4th International Workshop on Security and Dependability of Critical Embedded Real-Time Systems (CERTS 2019). Open Access Series in Informatics (OASIs), vol. 73, pp. 4:1–4:13. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany (2019) (Cited on page 63.)

Appendix

1 TinyConsole Makefile

```
1 OPENRSDK_ROOT?=/usr/local/OPEN_R_SDK
2 INSTALLDIR=../MS
3 CXX=$(OPENRSDK_ROOT)/bin/mipsel-linux-g++
4 STRIP=$(OPENRSDK_ROOT)/bin/mipsel-linux-strip
5 MKBIN=$(OPENRSDK_ROOT)/OPEN_R/bin/mkbin
6 STUBGEN=$(OPENRSDK_ROOT)/OPEN_R/bin/stubgen2
7 MKBINFLAGS=-p $(OPENRSDK_ROOT)
8 LIBS=-L$(OPENRSDK_ROOT)/OPEN_R/lib -lObjectComm -lOPENR -lInternet -
    lantMCOOP
9 CXXFLAGS= \
10 -O2 \ // gcc will try to reduce code size and execution time performing
11      // all supported optimisations that do not involve space-speed
12      // tradeoff (optimise more than O1)
13 -g \ // Enable debugging
14 -I. \ // Used to specify directories for header files
15
16 -I$(OPENRSDK_ROOT)/OPEN_R/include/R4000 \ // Header files specific to the
    CPU
17 -I$(OPENRSDK_ROOT)/OPEN_R/include/MCOOP \ // Header files for ANT
18 -I$(OPENRSDK_ROOT)/OPEN_R/include // Folder with all other header files
19
20 CXXFLAGS+= -DOPENR_DEBUG // The OSYSDEBUG() command is made available
21
22 .PHONY: all install clean
23
24 all: tinyConsole.bin // Final compiled binary
25
26 %.o: %.cc
27     $(CXX) $(CXXFLAGS) -o $@ -c $^
28
29 TinyConsoleStub.cc: stub.cfg
30     $(STUBGEN) stub.cfg
31
32 tinyConsole.bin: TinyConsoleStub.o TinyConsole.o tinyConsole.ocf
33 // Links object files, the ocf file is necessary, others are optional
    depending
34 // on program needs
35     $(MKBIN) $(MKBINFLAGS) -o $@ $^ $(LIBS)
36     $(STRIP) $@
37
38 install: tinyConsole.bin
39     gzip -c tinyConsole.bin > $(INSTALLDIR)/OPEN-R/MW/OBJS/TINYCONS.BIN
```

2 DoInit and DoStart

```
1 OStatus
2 TinyConsole::DoInit(const OSystemEvent& event)
3 {
4     OSYSDEBUG(("TinyConsole::DoInit()\n"));
5 }
```

```

6 // Macros defined in <OPENR/core_macro.h>
7 NEW_ALL_SUBJECT_AND_OBSERVER; // Registers the necessary number of
  observers and subjects
8 REGISTER_ALL_ENTRY; // Registers the connection to services offered by
  other objects
9 SET_ALL_READY_AND_NOTIFY_ENTRY; // Registers the entry points for
  receiving messages
10
11 OpenPrimitives();
12   NewCommandVectorData();
13
14
15 OPENR::SetMotorPower(opowerON);
16 // Necessary to power on motors and must be called in this function
17 return oSUCCESS;
18 }
19
20 OStatus
21 TinyConsole::DoStart(const OSystemEvent& event)
22 {
23     // ..... //
24     // Initialisation of ANT stack //
25     // ..... //
26     if (subject[sbjMove]->IsReady() == true) {
27         double cur[NUM_JOINTS];
28         ReadCurrentPose(cur);
29         SetupInterpolation(cur, SLEEPING_POSE_3, STARTUP_COUNTER);
30         // Used to move the joints to the sleeping pose with a counter
  which indicates the time that this action takes, value must be adjusted
  to avoid any sudden movements; robot only be switched on when it is
  already in a seated position
31
32         SetJointGain(); // Position of this function is important as if it
  isn't called at the "good" time,
33         // joints won't move at all when receiving commands
34         motionState = MSTATE_STARTUP;
35         AdvanceInterpolation(STARTUP_COUNTER); // Generate the 1st 128ms block
  of joint movements
36         // Sends it to the motor subject sbjMove; once this has finished
  the scheduler will ask for the next frame
37     } else {
38         motionState = MSTATE_IDLE; // In the case the motor observer is not
  ready/has crashed -> avoid sending data which could cause an exception
39     }
40
41     // Macros defined in <OPENR/core_macro.h>
42     ENABLE_ALL_SUBJECT; // Enables the subjects of the core class
43     ASSERT_READY_TO_ALL_OBSERVER; // Sends ASSERT_READY to all connecting
  subjects (as previously defined)
44
45     return Listen();
46 }

```

3 Assignment of joint values

```

1 static const char* const JOINT_LOCATOR[TinyConsole::NUM_JOINTS] = {
2     "PRM:/r4/c1-Joint2:41", // Right Front J1
3     "PRM:/r4/c1/c2-Joint2:42", // Right Front J2

```

```

4     "PRM:/r4/c1/c2/c3-Joint2:43",    // Right Front J3
5
6     "PRM:/r2/c1-Joint2:21",          // Left Front J1
7     "PRM:/r2/c1/c2-Joint2:22",       // Left Front J2
8     "PRM:/r2/c1/c2/c3-Joint2:23",    // Left Front J3
9
10    "PRM:/r3/c1-Joint2:31",           // Left Rear J1
11    "PRM:/r3/c1/c2-Joint2:32",        // Left Rear J2
12    "PRM:/r3/c1/c2/c3-Joint2:33",     // Left Rear J3
13
14    "PRM:/r5/c1-Joint2:51",           // Right Rear J1
15    "PRM:/r5/c1/c2-Joint2:52",        // Right Rear J2
16    "PRM:/r5/c1/c2/c3-Joint2:53",     // Right Rear J3
17 };
18
19 static const double SLEEPING_POSE_0[12] = {
20     10, 1, 30, // Front
21     10, 1, 30, // Front
22     -35, 5, 60, // Rear
23     -35, 5, 60, // Rear
24 };

```